POSTASSIST LK · J26-IT-372 · INDIVIDUAL COMPONENT

Smart Registered Mail Intake and Tracking

The complete guide in simple words – what every part of the system does, how the parts work together, and why each rule exists. No technical background needed.

WHAT IS INSIDE

1 The big picture and one letter's journey

2 Technical words in plain language

3 At the counter: accepting a letter

4 After the counter: tracking scans

5 The phone app and offline work

6 Messages to the sender

7 The sender's tracking page

8 Supervision pages for the postmaster

9 The IoT scale and camera

10 The "smart" parts (AI models)

11 Safety, privacy and reliability

12 Where the data is kept

13 How to start and use the system

14 The research side and test results

15 The teammate's KPI desk (shared server)

A–C Every file · every message · viva questions

HOW TO READ THIS GUIDE

Read Parts 1 and 2 first – they give the whole picture and explain the words. Then read Part 3 (the counter), because it is the heart of the component. The other parts can be read in any order. Grey boxes marked **In simple words** give the short version; green boxes give **examples**; amber boxes explain **why** a rule exists.

Component owner

Senarathna W.L.S.K. · IT23324138

B.Sc. (Hons) in Information Technology, SLIIT

Edition

English · October 2026

A Sinhala edition with the same content is provided separately

CONTENTS

What this guide covers

1 The big picture
The problem, the people, the five building blocks, one letter’s journey

2 Technical words in plain language
Frontend, backend, API, database, token, OCR, model, idempotent …

3 At the counter: accepting a letter
Photo, address reading, fields, weight, price, payment, label and receipt

4 After the counter: tracking scans
The nine statuses, who may scan what, the eight checks, GPS location rules

5 The phone app (PostAssist Field)
Screens, delivery proof, how offline saving and syncing work

6 Messages to the sender
When messages are made, the outbox, retries, languages

7 The sender’s tracking page
The secret link and what the public can and cannot see

8 Supervision and the other web pages
Dashboard, items, review queue, messages, offices, reports, audit

9 The IoT scale and camera
Parts, wiring, weighing, tare and calibration, firmware, camera

10 The “smart” parts (AI)
Script identification, handwriting reader, address parser, results

11 Safety, privacy and reliability
Passwords, roles, audit log, no double records, atomic saving

12 Where the data is kept
The MongoDB collections and files, in plain words

13 How to start and use the system
Start-up, addresses and ports, logins, the device, tests

14 The research side
Objectives SO1–SO4, experiments, acceptance tests, limitations

15 The teammate’s Postal KPI desk
Why another app shares the same server

A Every file, explained
Backend, web app, phone app, device, AI, tools, tests, documents

B Messages you may see, and what to do
Weight, price, scan, photo and tariff messages

C Questions an examiner may ask
Short, honest answers

PART 1

The big picture

What the component is for, who uses it, and how its pieces fit together.

1.1 What problem does this component solve?

Imagine **Nimal** walks into the Malabe post office to send a **registered letter** to his aunt in Jaffna. A registered letter is a letter that the post office tracks and gets signed for at delivery, so it costs more than an ordinary letter. Today four things can go wrong:

1. **Retyping.** The officer copies the whole address from the envelope into a computer by hand. It is slow and typing mistakes happen.
2. **Weight and price mistakes.** The officer reads the weight from a scale, looks it up in a price chart and types the amount. Each copy is a chance for an error.
3. **Weak evidence.** Later, staff record "arrived" or "delivered", but nothing shows where or how that scan really happened.
4. **Lost messages.** If the network or the SMS/e-mail service fails, Nimal never hears about his letter.

This component fixes each of these:

Problem	What the system does instead
Retyping the address	Takes a **photo** of the envelope and **reads the address** (printed or handwritten, Sinhala, Tamil or English). The officer only checks and corrects.
Weight and price mistakes	A **smart scale** sends the exact weight straight to the computer. The computer **calculates the price** from the official 2026 tariff and shows every line of the bill.
Weak evidence for scans	Every scan records **who**, **when** and **where** (GPS with its accuracy). The system checks whether the place makes sense and holds suspicious scans for a supervisor.
Lost messages	Messages are saved in an **outbox** at the same moment as the scan and are **retried** until they go out. Phones keep working **offline** and upload later.

> **IN SIMPLE WORDS**
>
> The component is a complete computer system for **accepting** registered letters at a post-office counter and **tracking** them until they are delivered – with a camera that reads addresses, a scale that weighs letters, phones that record deliveries, and e-mails that keep the sender informed.

Scope (what it covers): domestic registered letters of **5–500 g** at one pilot counter, a few approved offices and one delivery team, using **synthetic (practice) mail** until the university ethics committee and the Department of Posts give approval. **Not covered:** parcels, international mail, cash reconciliation (another member's component) and route planning.

1.2 The people who use it

Every person logs in with a **role**. The role decides which pages they see and what they are allowed to do. The demo accounts (created automatically) are shown in brackets.

Role	Who that is	What they do in the system	Uses
Counter officer	Officer at the post-office counter (`counter1` , `counter2`)	Accepts letters: photo, address, weight, price, payment, label, receipt. Can record “dispatched”.	Web app
Hub officer	Staff at a sorting centre / mail exchange (`hub1` , `hubjaf` , `hubdemo`)	Records dispatch, in-transit and arrival scans as letters move between offices.	Web app or phone
Postman	Delivers letters (`postman1` , `postman2`)	Records out for delivery, delivery attempted and delivered – with the recipient’s signature.	Phone app
Postmaster	Supervisor of an office (`pm1` , `pm2`)	Reviews held scans, watches failed messages, assigns postmen, sets office zones, downloads reports, reads the audit log.	Web app
Administrator	Technical staff (`admin`)	Everything, plus creating users and devices, approving tariffs and switching the location policy.	Web app
Sender	Member of the public (Nimal)	Gets a receipt, e-mail updates and a private tracking link. Never logs in.	Receipt, e-mail
Recipient	The person the letter is for	Signs on the postman’s phone, or gives their name.	Postman’s phone
IoT device	The smart scale itself (`SCALECAM-01`)	Reports the weight twice a second, takes photos, obeys “tare”, “calibrate” and “capture” commands. It logs in with its own secret key.	–

1.3 The five building blocks

The system is made of five pieces that talk to each other over the network. Think of a **restaurant**:

- The **web app** (also called the *frontend*) is the dining room and the menu – what staff see and click on in a web browser such as Chrome or Edge. It is built with **React**.
- The **phone app** is a waiter who also works outside, even where there is no signal. It is built with **React Native** and **Expo**.
- The **backend** (the *server*) is the kitchen. It does all the real work and enforces every rule. It is written in **Python** with **FastAPI**.
- The **database** is the store room and the record book. Everything is kept there. It is **MongoDB**.
- The **IoT scale and camera** is a special machine in the kitchen that weighs and photographs letters and reports to the backend. It is an **ESP32-CAM** board with a load cell.

Inside the kitchen there are also **specialist cooks**: the **AI models** that recognise the script (Sinhala, Tamil or English), read handwriting and split an address into its parts.

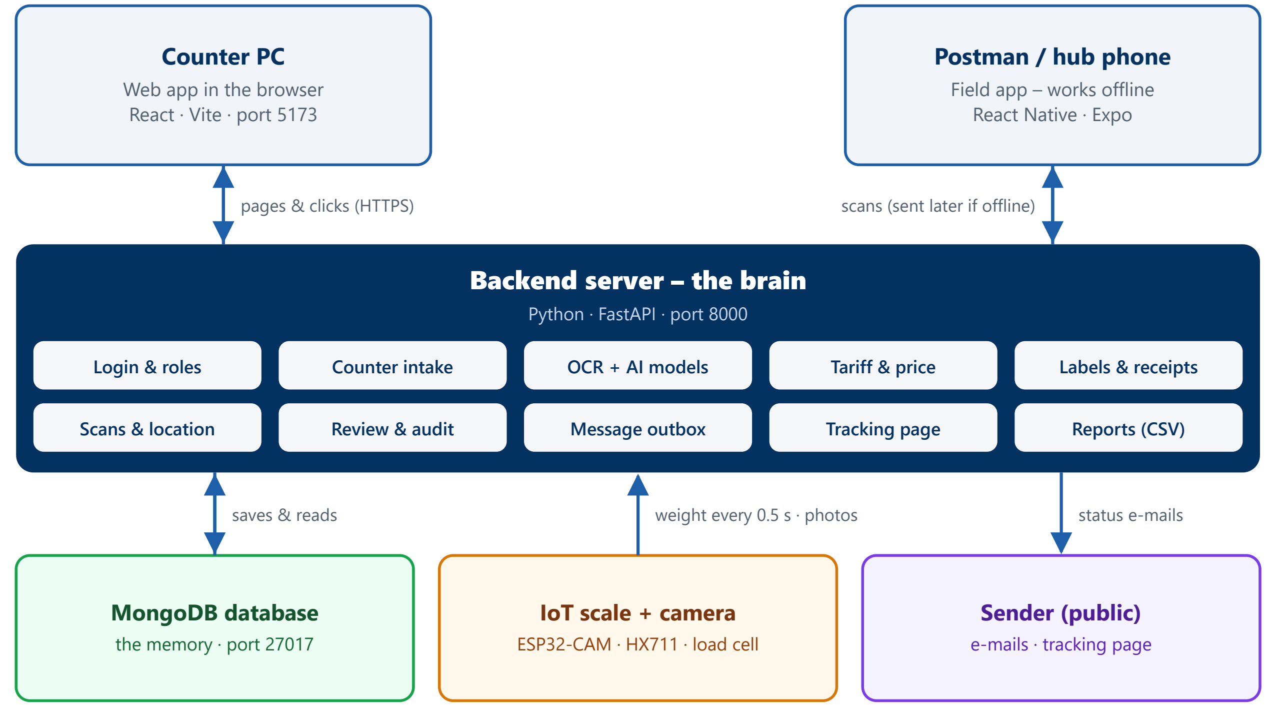

Counter PC
Web app in the browser
React · Vite · port 5173
Postman / hub phone
Field app – works offline
React Native · Expo
pages & clicks (HTTPS)
scans (sent later if offline)
Backend server – the brain
Python · FastAPI · port 8000
Login & roles
Counter intake
OCR + AI models
Tariff & price
Labels & receipts
Scans & location
Review & audit
Message outbox
Tracking page
Reports (CSV)
saves & reads
weight every 0.5 s · photos
status e-mails
MongoDB database
the memory · port 27017
IoT scale + camera
ESP32-CAM · HX711 · load cell
Sender (public)
e-mails · tracking page

Figure 1 – The five building blocks and what travels between them. Everything goes through the backend; the web app, the phone and the scale never talk to the database directly.

WHY EVERYTHING GOES THROUGH THE BACKEND

If the web page or phone could change the database directly, anyone could skip the rules (for example, mark a letter delivered without a signature). Because every request passes through the backend, every rule is checked in one place, every time.

1.4 One letter's journey, from counter to doorstep

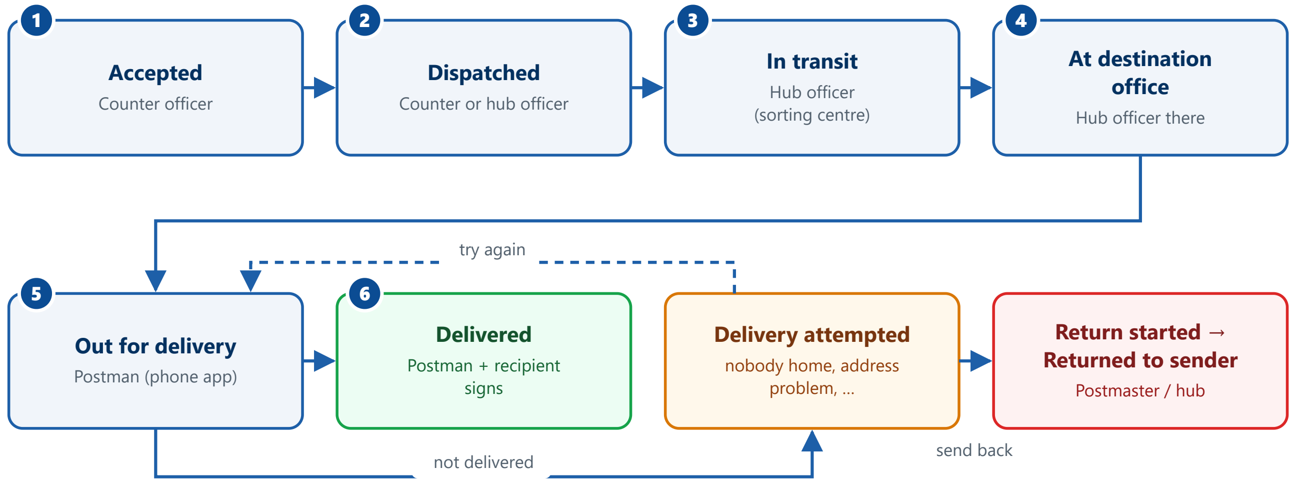

1
Accepted
Counter officer
2
Dispatched
Counter or hub officer
3
In transit
Hub officer
(sorting centre)
4
At destination office
Hub officer there
try again
5
Out for delivery
Postman (phone app)
6
Delivered
Postman + recipient signs
Delivery attempted
nobody home, address problem, ...
Return started →
Returned to sender
Postmaster / hub
not delivered
send back

Figure 2 – The normal journey (1–6) and the two side roads: a failed delivery attempt, and a return to the sender.

1. **At the counter (Malabe).** The officer starts a new intake, puts the envelope on the scale and takes a photo. The system reads the address; the officer checks and confirms it. The scale weight is used, the system calculates **Rs 130.00**, Nimal pays, and the officer clicks **Accept**. The letter becomes an **item** with a tracking ID such as `PA-DEMO-7K3M9Q2XRH4`. A label and a receipt are printed. Nimal gets an e-mail: “accepted”.
2. **Dispatch.** The letter leaves the Malabe office. A scan records “Dispatched”.
3. **In transit.** At the Central Mail Exchange in Colombo a hub officer scans it: “In transit”.
4. **Arrival.** At Jaffna main post office it is scanned “Arrived at destination office”. The phone’s GPS position is compared with the office’s approved area.
5. **Out for delivery.** The postman scans it before leaving. Nimal gets an e-mail.
6. **Delivered.** The aunt signs on the postman’s phone. The scan “Delivered” is saved with the signature. Nimal gets an e-mail.

At any time Nimal can scan the QR code on his receipt to open a **tracking page** showing this timeline. If something looks wrong – for example the GPS shows the postman far from the delivery area, or there is no signature – the scan is **held** until the postmaster checks it.

PART 2

Technical words in plain language

The words used in the rest of this guide – and in your viva – explained without jargon.

2.1 How software is organised

Word	Meaning in simple words
Frontend	The part people see and click on – screens, buttons, forms. Here: the web app and the phone app.
Backend / server	A program running on a computer that receives requests, applies the rules, talks to the database and sends answers back. Users never see it directly.
Browser	Chrome, Edge, Firefox … the program that shows web pages.
API	“Application Programming Interface” – the set of doors (like the kitchen’s order window) through which the frontend, the phone and the scale ask the backend to do things. Our API has about 70 doors.
Endpoint	One single door of the API, written like `POST /api/intake/drafts` (“create a new draft”).
Request / response	A question sent to an endpoint, and the answer that comes back.
GET · POST · PATCH	The kind of request: GET = read something, POST = create or do something, PATCH = change part of something.
JSON	The text format used to send data, e.g. `{"grams": 20.4, "stable": true}`.

Word	Meaning in simple words
Status code	A number in every answer: 200 = OK, 201 = created, 401 = not logged in, 403 = not allowed, 404 = not found, 409 = conflict with the current state, 422 = something missing or invalid, 503 = a service is unavailable.
Database	The permanent store of data. Ours is **MongoDB**. Data survives when programs stop.
Collection · document	MongoDB words. A *collection* is like a drawer (e.g. “items”); a *document* is one record in it (one registered letter).
localhost · IP address · port	*localhost* means “this same computer”. An *IP address* (e.g. `172.20.10.5`) is a computer’s address on the network. A *port* is a numbered door on that computer: 8000 = backend, 5173 = web app, 27017 = MongoDB.
HTTP / HTTPS	The language browsers and servers speak. HTTPS is the encrypted version (the padlock).
Firmware	The program that runs inside a small device such as the ESP32 scale.
Repository · commit	The project folder kept with **git**, which remembers every saved change (*commit*) and who made it.

2.2 Words about accounts and safety

Word	Meaning in simple words
Role	The job a user has (counter officer, postman …). It decides what they may do.
Password hash	A scrambled, one-way version of a password. The system stores only this, so even someone who reads the database cannot see passwords.
Login token (JWT)	After you log in, the backend gives the browser a signed “wristband”. The browser shows it with every request. It expires after 12 hours.
Device key	The scale’s own secret password, sent with every reading.
Audit log	A diary that can only be added to: who did what, to which record, when and why.
Idempotent	Doing it twice has the same effect as doing it once – like pressing a lift button ten times: the lift still comes once. Accepting the same draft twice gives the same single item.
Atomic	All or nothing. A scan, the new status and the sender’s message are saved together in one step, or not at all.
Outbox	A tray of messages waiting to be sent, kept inside the database.

2.3 Words about mail, devices and AI

Word	Meaning in simple words
Draft	An intake that is still in progress at the counter. It saves itself and can be resumed.
Item	A letter that has been accepted and paid for. It has a tracking ID.
Tracking ID	The letter’s unique number, e.g. `PA-DEMO-7K3M9Q2XRH4`.

Word	Meaning in simple words
Event / scan	One recorded step in the letter's journey ("dispatched", "delivered" …) with time, person and location.
Barcode (Code 128) · QR code	Two kinds of printed code that a scanner or phone camera can read. Ours contain only the tracking ID (or the tracking link).
Tariff · band	The official price list. A *band* is a weight range with one price, e.g. "more than 0 g up to 30 g = Rs 70".
IoT	"Internet of Things" – everyday objects (here, a scale) connected to the network.
Microcontroller (ESP32)	A tiny computer on one chip, with Wi-Fi. Ours also has a camera.
Load cell · HX711	A metal bar that bends very slightly under weight, and the small chip that measures that bending.
Tare · calibrate	*Tare*: tell the scale "this is zero". *Calibrate*: put a known weight on it so it learns how much bending equals one gram.
GPS fix · accuracy	A position from the phone's GPS, and the phone's own estimate of how far off it might be (a radius in metres).
Offline · sync	Working without network, then sending the saved work when the network returns.
OCR	"Optical Character Recognition" – turning a photo of text into typed text.
Tesseract	A free, well-known OCR program for *printed* text, with language packs for Sinhala (`sin`), Tamil (`tam`) and English (`eng`).
Machine-learning model	A program that learned a task from many examples instead of being given rules by hand. *Training* = the learning; the result is saved in a model file.
ONNX	A standard file format for trained models, so they run quickly on an ordinary laptop without the training software.
Confidence	How sure a model is about its answer, from 0 % to 100 %.
CER / WER	Character / word error rate: the share of characters (or words) a reader got wrong. Lower is better.
Gazetteer	A list of real places: Sri Lankan towns, villages, districts, postcodes and post offices (here, from OpenStreetMap).

2.4 The tools the component is built with

Tool	What it is	Used for
Python 3.14	Programming language	Backend, AI training
FastAPI	Python library for building APIs; it also creates the API documentation page `/docs`	Backend
MongoDB 8	Document database	All stored data
JavaScript · React 19	Language and library for interactive web pages	Web app

Tool	What it is	Used for
Vite · Tailwind CSS	Vite runs and packages the web app; Tailwind gives it the navy look	Web app
React Native · Expo	Build phone apps with React; Expo Go runs them without an app store	Phone app
Arduino C++	Language for the ESP32 firmware	Scale + camera
PyTorch · ONNX Runtime	Train AI models on the GPU · run them on any CPU	AI models
OpenCV · Tesseract	Image processing · printed-text OCR	Reading envelopes
python-crfsuite	Library for the CRF model	Address parser

PART 3

At the counter: accepting a letter

The heart of the component. Every button and every rule from login to the printed receipt.

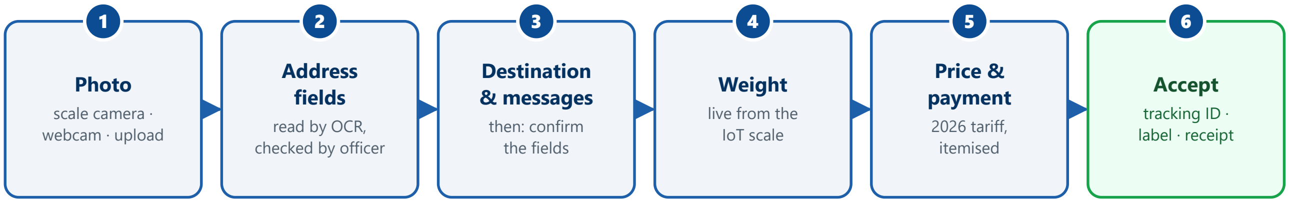

1
Photo
scale camera · webcam · upload
2
Address fields
read by OCR, checked by officer
3
Destination & messages
then: confirm the fields
4
Weight
live from the IoT scale
5
Price & payment
2026 tariff, itemised
6
Accept
tracking ID · label · receipt

Figure 3 – The six steps of a counter intake.

3.1 Logging in

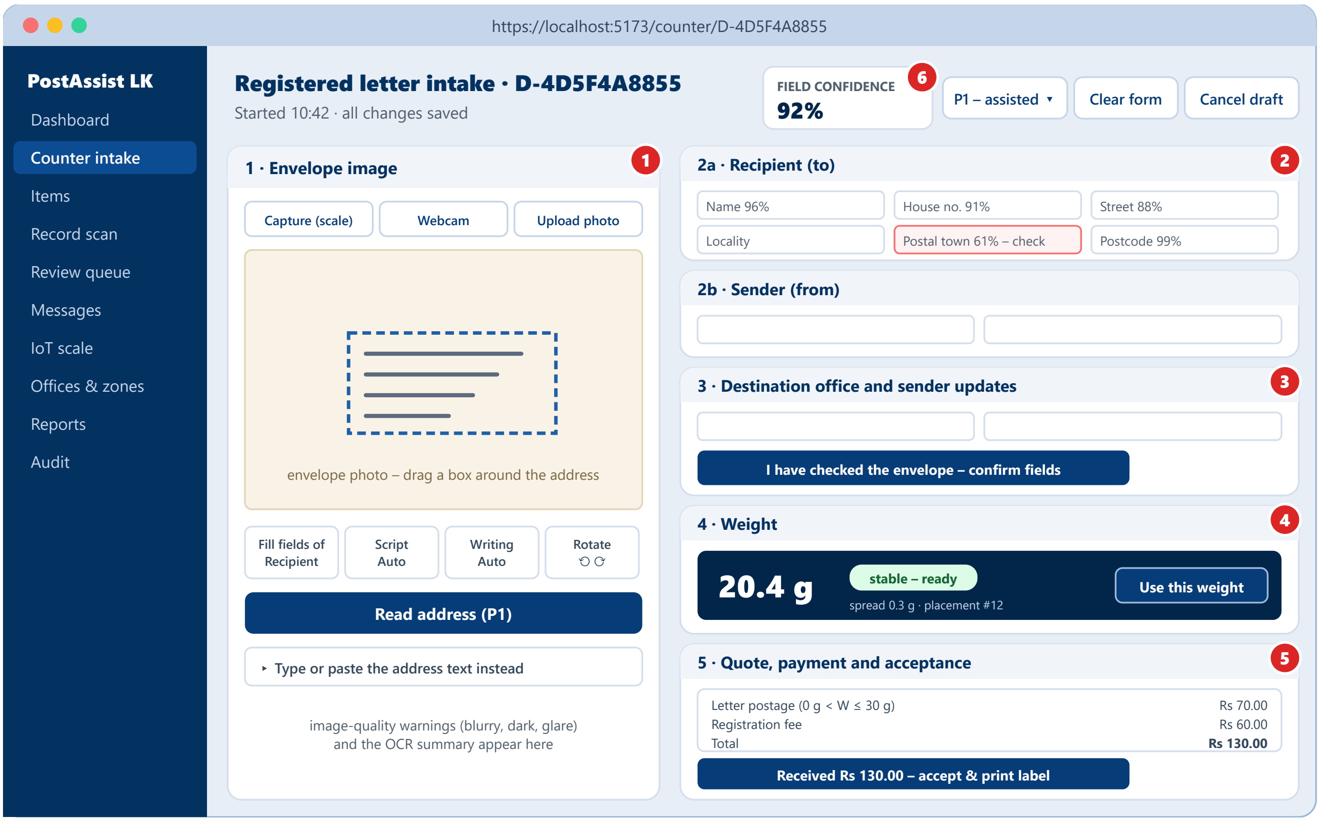
The officer opens the web app and types a username and password. The backend compares the password with its stored hash. If it is wrong, the answer is “wrong username or password” and a **security entry** is written to the audit log. If it is right, the browser receives a login token for 12 hours. Counter officers go straight to the **Counter intake** page; everyone else starts at the **Dashboard**.

3.2 The Counter intake page

- **New registered letter** – creates a new *draft* with an ID like `D-4D5F4A8855` and opens the intake screen.
- **Capability badges** – small coloured labels showing what is available right now: *OCR: Tesseract 5.5* (or “not installed – type/paste text”), missing language packs, *Script ID model loaded*, *Handwriting model loaded*, *Address parser: crf* (or “rules” if the model file is missing) and *LLM normaliser: none*. Green = ready, amber = missing.
- **Unfinished drafts (resume)** – every draft that was not finished, newest first, with its ID, recipient name and town, the experiment configuration (B0/B1/P1), “fields confirmed” and weight badges and the last change time. Click the ID to continue where you stopped.

- **Delete** – removes an unfinished draft from the list without opening it. It asks for confirmation, then marks the draft **cancelled** and frees its scale reading. The draft is not erased from the database – the audit log keeps a record that it was cancelled and by whom.

3.3 The intake screen at a glance



https://localhost:5173/counter/D-4D5F4A8855

Registered letter intake · D-4D5F4A8855

Started 10:42 · all changes saved

FIELD CONFIDENCE 92%

Letter postage (0 g < W ≤ 30 g) Rs 70.00
Registration fee Rs 60.00
Total Rs 130.00

PostAssist LK
Dashboard
Counter intake
Items
Record scan
Review queue
Messages
IoT scale
Offices & zones
Reports
Audit
6
P1 – assisted ▾
Clear form
Cancel draft
1 · Envelope image
1
Capture (scale)
Webcam
Upload photo
envelope photo – drag a box around the address
Fill fields of Recipient
Script Auto
Writing Auto
Rotate ⟲ ⟳
Read address (P1)
‣ Type or paste the address text instead
image-quality warnings (blurry, dark, glare) and the OCR summary appear here
2a · Recipient (to)
2
Name 96%
House no. 91%
Street 88%
Locality
Postal town 61% – check
Postcode 99%
2b · Sender (from)
3 · Destination office and sender updates
3
I have checked the envelope – confirm fields
4 · Weight
4
20.4 g
stable – ready
spread 0.3 g · placement #12
Use this weight
5 · Quote, payment and acceptance
5
Received Rs 130.00 – accept & print label

Figure 4 – Sketch of the intake screen. Red numbers: ① envelope image, ② recipient and sender fields, ③ destination office and sender updates, ④ weight, ⑤ price, payment and acceptance, ⑥ experiment configuration (B0/B1/P1), Clear form and Cancel draft.

- **Autosave**: about 0.6 seconds after you stop typing, the draft is saved ("all changes saved"). You can close the browser and resume later.
- **Field confidence** tile: the average confidence of the address parser over the recipient fields it suggested, with a *needs review* or *no flags* badge and a purple *handwriting model* badge when the handwriting reader was used.
- **Clear form** empties the sender, recipient, destination and contact fields (after asking). **Cancel draft** cancels the whole draft and returns to the counter page.

3.4 Step 1 – the envelope photo

There are three ways to get a photo:

Button	What happens
Capture from scale camera	The backend queues a *capture* command for the scale. The scale picks it up within half a second, photographs the envelope lying on its platform and uploads the JPEG. If no photo arrives within 25 seconds, the request expires.

Button	What happens
Webcam	Uses the computer’s camera. Browsers allow this only on `https://` or `localhost` addresses.
Upload photo	Choose a JPEG or PNG file (up to 8 MB), for example a phone photo.

Quality check. Every image is measured immediately. If there is a problem, an amber box lists it with advice:

Warning	Meaning (and the measured limit)
`LOW_RESOLUTION`	Image too small – the short side is below 600 pixels. Move closer or use a higher resolution.
`BLURRY`	Not sharp enough (sharpness score below 60). Hold still or refocus.
`TOO_DARK` / `TOO_BRIGHT`	Average brightness below 60 or above 225 (on a 0–255 scale).
`LOW_CONTRAST`	The text hardly stands out from the paper.
`GLARE`	More than 6 % of the image is pure white shine. Tilt the envelope or switch the flash off.

Crop and rotate. Drag a box around the address block on the photo; use *Rotate left / right* if the envelope is sideways. The reader uses exactly this crop and rotation.

Reading options. *Fill fields of* (Recipient or Sender) · *Script* (Auto-detect, Sinhala, Tamil, English) · *Writing* (Auto, Handwritten, Printed). Then press **Read address**. Or open *Type or paste the address text instead* and press **Split into fields** – the same parser then works on typed text.

3.5 How the address is read (the P1 pipeline)

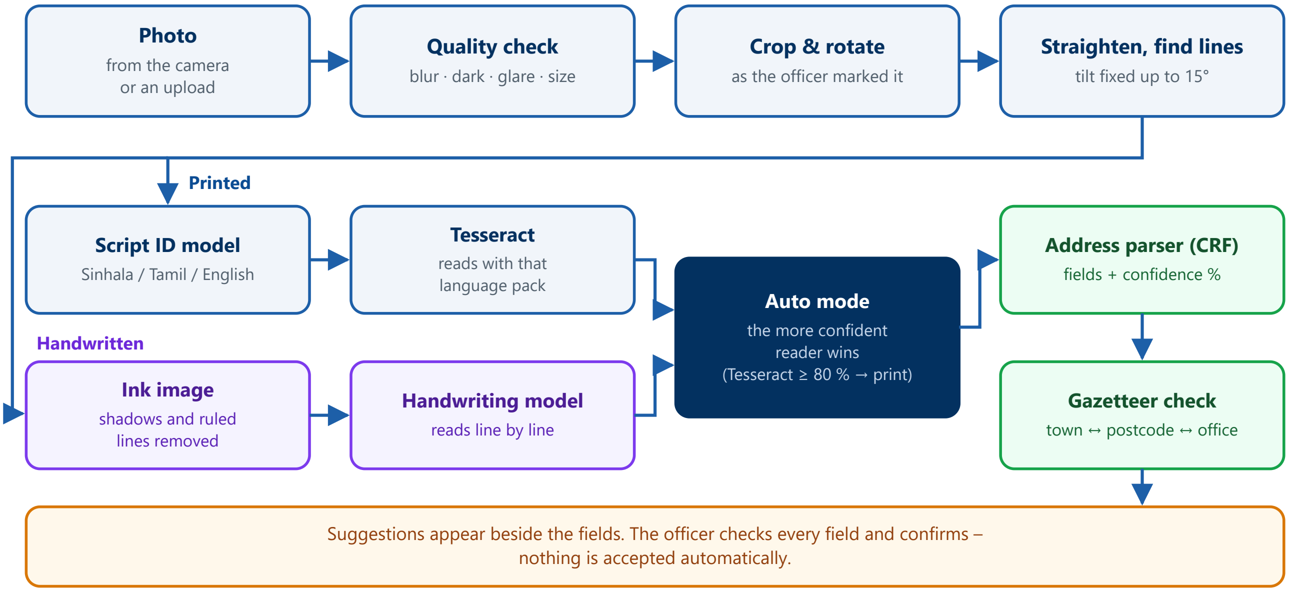


Photo
from the camera or an upload
Quality check
blur · dark · glare · size
Crop & rotate
as the officer marked it
Straighten, find lines
tilt fixed up to 15°
Printed
Script ID model
Sinhala / Tamil / English
Tesseract
reads with that language pack
Handwritten
Ink image
shadows and ruled lines removed
Handwriting model
reads line by line
Auto mode
the more confident reader wins
(Tesseract ≥ 80 % → print)
Address parser (CRF)
fields + confidence %
Gazetteer check
town ↔ postcode ↔ office
Suggestions appear beside the fields. The officer checks every field and confirms – nothing is accepted automatically.

Figure 5 – The assisted-capture pipeline (configuration P1). Printed and handwritten text take different roads and meet again at the address parser.

1. **Quality check** – as above.
2. **Crop and rotate** – exactly as the officer marked.

3. **Straighten** – if the text lines are tilted (between 0.3° and 15°) the image is turned level.
4. **Find the lines** – the system finds each line of writing.
5. **Printed road** – the **script identification model** looks at each line and decides Sinhala, Tamil or English; then **Tesseract** reads the block with that language pack (plus English for mixed addresses).
6. **Handwritten road** – the photo is turned into a clean **ink image** (shadows, uneven light and faint ruled lines removed) and the **handwriting model** reads it line by line.
7. **Auto mode** – if Tesseract is very sure (average word confidence 80 % or more) the print reading is used. Otherwise the handwriting model also reads the block and **the more confident reader wins**.
8. **Address parser** – the text is split into fields (name, house number, street, locality, postal town, postcode, district/province, country), each with a **confidence**.
9. **Flags** – a field whose confidence is below the frozen review threshold, or that was built from a handwritten word the reader was unsure of, is shown in **red**: "Low confidence – check against the envelope".
10. **Gazetteer check** – the town and postcode are compared with the list of real places and post offices.
11. **Suggestions only** – suggestions fill only **empty** fields. Anything the officer already typed is never overwritten. Under each field the original suggestion stays visible ("Suggested: … use").

For the research comparison the officer can choose a **configuration** per draft:

Config	What it means
B0	Manual entry only – the officer types from the photo (the baseline "today" situation).
B1	Basic OCR: Tesseract with all three language packs at once, simple hand-written rules to split the fields.
P1	The proposed assisted capture: quality check, straightening, script identification, handwriting model, trained parser with confidence, gazetteer checks.

WHY THE OFFICER MUST ALWAYS CONFIRM

Even a good reader makes mistakes – roughly one handwritten word in two may contain an error. The system **saves typing**; it does not replace reading the envelope. That is why OCR output is kept separately as a "suggestion" and nothing is accepted until the officer confirms.

3.6 Step 2 – recipient and sender fields

Each address block has seven boxes: **Name · House / unit no. · Street / road · Village / locality · Postal town · Postcode · District / province**. Next to a suggested value a small badge shows the model's confidence (e.g. 96%); low ones are red. **Use all suggestions** copies every suggested value at once.

- **Town / postcode check** – half a second after you type, the system checks the pair against the gazetteer: ✓ agree , mismatch (with the expected postcode) or unknown .
- **Derived postcode** – for Colombo zones (e.g. "Colombo 03") the postcode can be worked out, but it is only filled in when the officer clicks **Use it**. The system never inserts information silently.

- **Required:** for both sender and recipient – a name, a postal town, and at least one of house number / street / locality.

3.7 Step 3 – destination office and sender updates

- **Destination delivery office** – the office that will deliver the letter. When the recipient's postcode or town matches exactly one office, it is chosen automatically.
- **Send status updates by** – *No messages (receipt only)*, *E-mail*, or *SMS* (only if an SMS gateway is approved). The contact is checked: a valid e-mail address, or a Sri Lankan mobile number `07XXXXXXXX` / `+947XXXXXXXX`.
- **Message language** – English, සිංහල or தமிழ்.
- **Which updates** – *Key updates only* (accepted, out for delivery, attempted, delivered, return) or *Every accepted scan* (adds dispatched, in transit and arrival).

"I have checked the envelope – confirm fields". This button checks all required fields. If something is missing, a list of problems appears. If everything is there, the draft is marked **confirmed** and the system records how the officer used the suggestions – how many were **kept**, **corrected**, **typed manually** or **removed**. These numbers are research data (they show how much typing OCR saved). If any field is edited after confirming, the confirmation is removed and must be given again.

3.8 Step 4 – weight

A dark panel shows the **live reading** from the scale, refreshed about every 0.7 seconds: the grams, a badge (**stable – ready** or what is wrong), the *spread* (how much the reading wobbled in the last second), the *placement number* and the calibration ID. The button **Use this weight** works only when **all** of these are true:

Rule	Why
The reading is **fresh** – at most 3 seconds old	An old reading may belong to a different letter or a disconnected scale.
The **sensor works** (HX711 answering)	A broken wire would give nonsense.
The scale has a **zero point** (it is not waiting for a tare)	Without a zero point, the number means nothing. Only an older bridged board loses its zero on restart.
The scale is **calibrated**	Otherwise it does not know what one gram is.
Something is **on the platform** (more than 2 g)	You cannot weigh air.
The reading is **stable** – it moved less than 1 g during the last second	A letter still swinging gives a wrong value.
The weight is **not negative**	A negative value means the scale needs taring.
This **placement** was not used for another letter	Stops the previous letter's weight from being reused for a new one.

IN SIMPLE WORDS – THE PLACEMENT NUMBER

Every time the platform becomes empty and something new is put on it, the scale counts up: placement #11, #12, #13 … Each placement can be paired with **only one** letter. If the officer forgets to take the old letter off, the system says “this reading belongs to item PA-DEMO-… – lift the letter off the scale and place the new letter”.

Uncertainty. Each paired weight carries a “±” value – the *expanded uncertainty*. By default it is ±2 g; after weighing trials the postmaster can enter the measured value. It is never smaller than half of the observed wobble.

Manual weight (audited). If the scale is broken or the letter is heavier than 500 g, the officer opens *Manual weight from an approved instrument*, types the grams, the instrument (e.g. “Counter scale #2”) and a reason of at least 5 characters. This is saved with a **warning** entry in the audit log. Manual weights are allowed up to 2,000 g (the tariff maximum).

3.9 Step 5 – the price (quote)

THE FORMULA

Charge = letter postage for the weight band + registration fee Rs 60 + (optional) advice of delivery Rs 100 + (optional) insurance Rs 100 for every Rs 1,000 (or part) of declared value, up to Rs 25,000.

The tariff is version `SLP-DOM-2026-02-09`, transcribed from the Department of Posts notice “Domestic postage revision effective 9 February 2026”. A weight *W* belongs to the band where *lower* < *W* ≤ *upper*.

Weight up to (g)	30	40	50	60	70	80	90	100	200	300	400	500
Postage (Rs)	70	80	90	100	110	120	130	140	160	180	200	220
Registered total (Rs)	**130**	140	150	160	170	180	190	200	220	240	260	280

The tariff continues above 500 g (600 g = Rs 240 postage … 2,000 g = Rs 440) for manual weights from an approved scale.

EXAMPLE 1 – AN ORDINARY LETTER

20.4 g ± 2 g → band “0 < W ≤ 30 g” → postage Rs 70.00 + registration Rs 60.00 = **Rs 130.00**.

EXAMPLE 2 – WITH INSURANCE

Declared value Rs 2,500 → 3 blocks of Rs 1,000 (the last one partial) → 3 × Rs 100 = **Rs 300** insurance on top.

THE BOUNDARY RULE – THE SYSTEM NEVER GUESSES

A letter of **29 g ± 2 g** could really weigh anything from 27 g to 31 g – on either side of the 30 g price step. Instead of quietly picking the cheaper or the dearer band, the system answers **“Reweigh required”** and asks for the approved scale or the approved exception procedure.

- Below 5 g → rejected (“check the scale is tared and the item is on the platform”). Above 500 g from the IoT scale → rejected (“weigh on the approved counter scale and enter it as a manual weight”).

- A quote is valid for **10 minutes**. It shows the tariff version and, until an administrator approves the tariff, the badge **transcribed – research use**.
- Tariff safety: exactly one tariff must be in force on the day, and its bands must follow each other without gaps or overlaps. Otherwise quoting stops with a clear code (see Appendix B).

3.10 Payment and acceptance

When the quote is OK, the officer chooses the payment method (cash, card, other), optionally types a receipt reference, and presses **Received Rs 130.00 – accept & print label**. The button stays disabled until the fields are confirmed. Before accepting, the backend checks again: fields confirmed, weight present, the quote is the latest one and not expired, and the payment equals the quoted total.

Then, in one step, it:

1. creates the **item** with a new tracking ID,
2. adds its first event, “accepted”,
3. puts an “accepted” message in the outbox (if the sender chose e-mail or SMS),
4. marks the draft as **committed**, and
5. writes an audit entry (tariff version, total, weight source).

> **WHY A DOUBLE CLICK IS HARMLESS**
>
> The database allows only one item per draft. If the network times out and the officer presses Accept again, the backend returns the **same** item – never a second letter or a second payment. This is acceptance test T05.

The tracking ID

`PA-DEMO-7K3M9Q2XRH4` = prefix **PA-DEMO** (prototype – not an official Sri Lanka Post number) + **10 random characters** + **1 check character**.

- The characters come from an alphabet without I, L, O and U, so they cannot be confused with 1 and 0. When an ID is typed, O is read as 0 and I/L as 1 automatically.
- The ID is random, not a running number, so nobody can guess a neighbour’s letter from their own receipt.
- The check character catches every single wrong character and every swap of two neighbouring characters when an ID is typed by hand.
- It contains no personal data.

3.11 Label and receipt

After acceptance the **label print** page opens:

- **Envelope label** – a large “R” (registered), the origin office and time, a **QR code** and a **Code 128 barcode**, the ID in large letters and the note “Prototype identifier – not an official postal number”. Both codes contain only the tracking ID.
- **Sender receipt** – tracking ID, when, where and by whom it was accepted, from / to, destination office, weight and where it came from (IoT scale or approved scale), every charge line, the total and payment

method, the tariff version, the tracking link and a QR code “Scan to track”.

- **Print label & receipt** – the first print is recorded. A **reprint** asks for a reason (e.g. “printer jam”); it is audited and never creates a new item or charge (test T06).
- **Verify** – the officer scans the printed barcode with a USB barcode reader into the box. If it reads back the same ID: “Label verified – place the letter in the dispatch tray”. If not: “LABEL_MISMATCH – reprint the label”.

PART 4

After the counter: tracking scans

How each step of the journey is recorded, who may record it, and how the system decides whether to trust a scan.

4.1 The nine statuses

Status	Meaning	Recorded by
Accepted	Accepted and paid at the counter.	Created automatically at acceptance
Dispatched	Left the origin office.	Counter or hub officer of the origin office
In transit	Handed over at a sorting centre on the way (can happen several times).	Hub officer
At destination office	Arrived at the office that will deliver it.	Hub officer of the destination office
Out for delivery	The postman has taken it out.	Postman of the destination office
Delivery attempted	The postman tried but could not deliver (a reason is required).	The assigned postman
Delivered	Handed over, with an acknowledgement (signature, name or delivery-sheet number).	The assigned postman
Return started	Being sent back to the sender (a reason is required).	Postmaster
Returned to sender	Back with the sender. Final.	Postmaster

4.2 Allowed moves

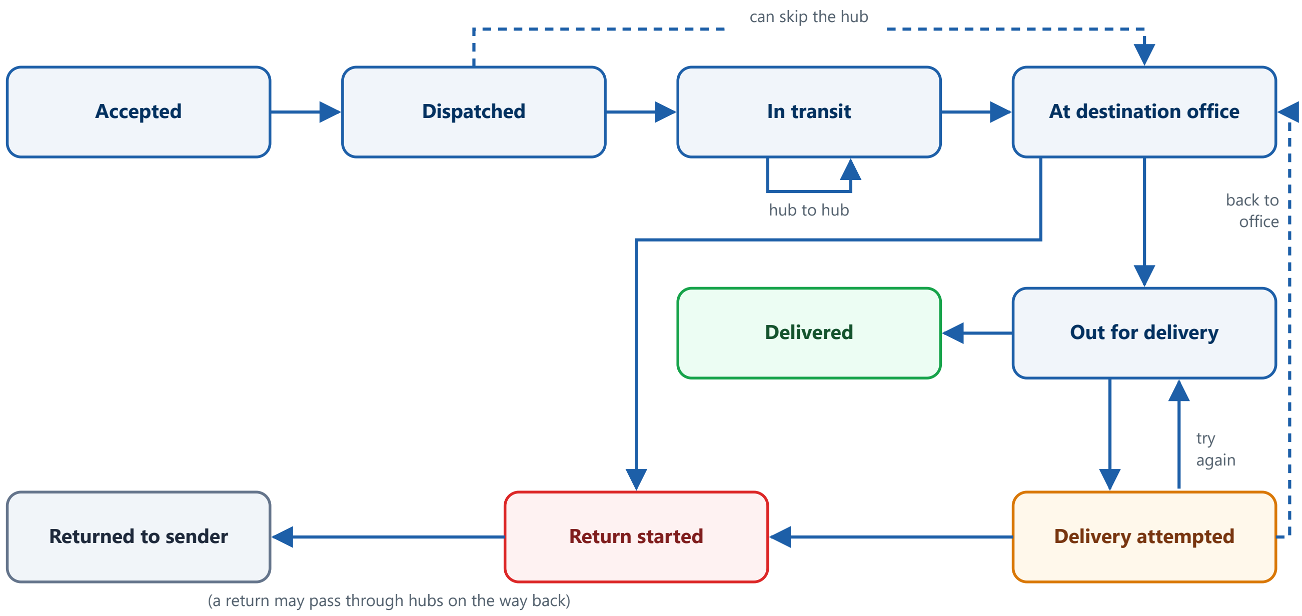

can skip the hub
Accepted
Dispatched
In transit
At destination office
hub to hub
back to office
Delivered
Out for delivery
try again
Returned to sender
Return started
Delivery attempted
(a return may pass through hubs on the way back)

Figure 6 – The only moves the system allows. “Delivered” and “Returned to sender” are final.

- A letter can never jump backwards or skip to an impossible state (e.g. from “accepted” straight to “delivered”). Such a scan is rejected with `INVALID_TRANSITION`.
- **Late scans.** If a phone was offline and uploads an *older* step after the letter has moved on (e.g. an “in transit” scan arrives after “delivered”), it is stored in the history only and does not change the current status (test T13).
- **Returns.** After “return started” the letter travels back; it may pass through hubs again and ends at “returned to sender”. Earlier delivery attempts stay in the history (test T17).

4.3 Who may record what

Role	Statuses	Extra rule about the letter (“object-level” rule)
Counter officer	dispatched	Only letters accepted at their own office.
Hub officer	dispatched, in transit, at destination office	Dispatch only at the origin office; arrival only at the destination office.
Postman	out for delivery, attempted, delivered	Destination office must be their office; postcode in their beat or the letter assigned to them; attempted/delivered only if assigned to them.
Postmaster	all scan statuses, including returns	Their office must be the origin or the destination.
Administrator	all	No office limit.

When a postman records “out for delivery”, the letter is automatically assigned to that postman. A postmaster can also assign letters on the item page.

4.4 How a scan is recorded

- **At a desk (web app, “Record scan” page)** – for counter and hub staff. Scan the label with a USB barcode reader (it types the ID) or type it, press **Look up**. The page shows the current status and **only the statuses you may record now**. Tick “Attach this computer’s location” if wanted, then **Record scan**.
- **On the road (phone app)** – for postmen and hub officers. Described in Part 5.

Each scan carries: a unique **event ID** made once on the device, the tracking ID, the requested status, the **capture time**, the device, and the **location** (latitude, longitude, accuracy, how old the fix is, permission state, and Android’s “mock location” flag). The worker’s identity is always taken from the login token, never from the scan itself.

4.5 The eight checks every scan goes through

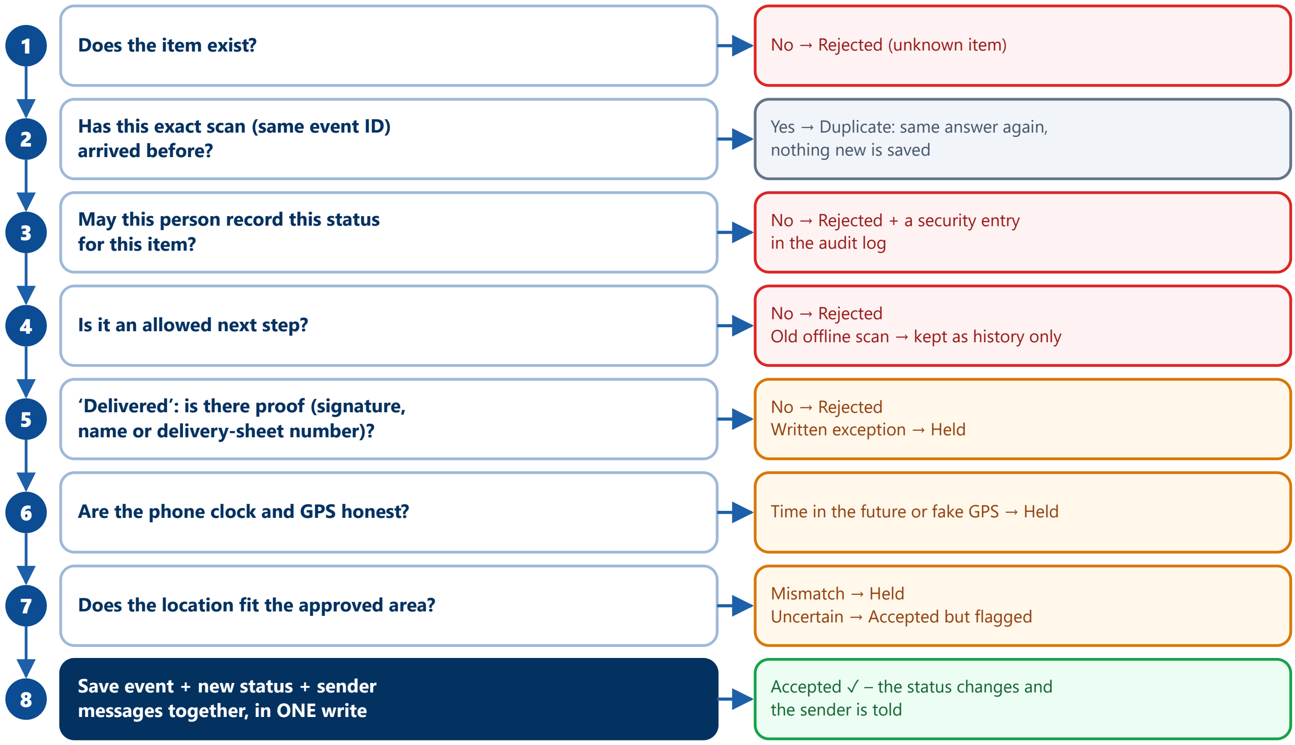

1
Does the item exist?
No → Rejected (unknown item)
2
Has this exact scan (same event ID) arrived before?
Yes → Duplicate: same answer again, nothing new is saved
3
May this person record this status for this item?
No → Rejected + a security entry in the audit log
4
Is it an allowed next step?
No → Rejected
Old offline scan → kept as history only
5
‘Delivered’: is there proof (signature, name or delivery-sheet number)?
No → Rejected
Written exception → Held
6
Are the phone clock and GPS honest?
Time in the future or fake GPS → Held
7
Does the location fit the approved area?
Mismatch → Held
Uncertain → Accepted but flagged
8
Save event + new status + sender messages together, in ONE write
Accepted ✓ – the status changes and the sender is told

Figure 7 – The decision pipeline. The order matters: a good GPS position can never make up for a missing permission or an impossible step.

Decision	What it means
accepted	Saved, the status changes, messages are queued.
accepted + flagged	Saved and applied, but marked for attention (e.g. location uncertain). Shown in the review page’s “flagged” list.
held for review	Saved but **does not change the status and sends no message** until a postmaster approves it.
rejected	Not saved into the item at all; only the audit log records the attempt.
duplicate	The same event ID was already saved – the original answer is returned again.

Decision	What it means
accepted / rejected *after review*	The postmaster’s final decision on a held scan.

If two people change the same letter at the same moment, the second save notices (each item has a **revision number**), re-reads the letter and checks again, so nobody overwrites anybody.

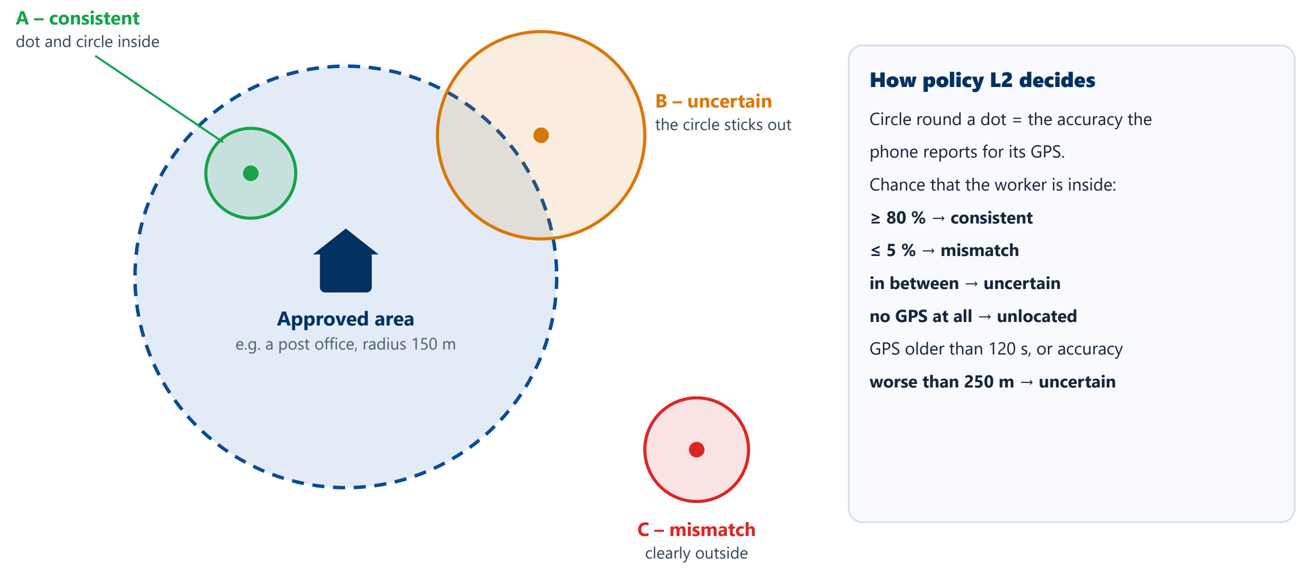
4.6 The location check

Each status is compared with an **approved reference area**:

Scan	Compared with
Dispatched	The origin office’s facility zone (e.g. 150 m radius).
In transit · return started	The scanning worker’s own office.
At destination office	The destination office’s facility zone.
Out for delivery	The destination office’s delivery area (e.g. 2–4 km radius).
Attempted · delivered	A confirmed recipient location (if a postmaster set one, with its source), otherwise the delivery area. A town centre is never treated as a house.
Returned to sender	The origin office.

Three **location policies** exist so they can be compared in the research:

- **L0 – status only:** the location is recorded but not judged (the baseline).
- **L1 – fixed radius:** inside the circle = consistent, outside = mismatch. It ignores how accurate the GPS is.
- **L2 – uncertainty-aware (default, `LOC-2026.10-L2`):** treats the GPS position as a fuzzy cloud whose size comes from the reported accuracy, and computes the **probability** that the worker is really inside the area.



How policy L2 decides

Circle round a dot = the accuracy the phone reports for its GPS.

Chance that the worker is inside:

≥ 80 % → consistent

≤ 5 % → mismatch

in between → uncertain

no GPS at all → unlocated

GPS older than 120 s, or accuracy **worse than 250 m → uncertain**

Figure 8 – Policy L2. A: clearly inside. B: the GPS is too vague to be sure. C: clearly outside.

Location result	What happens (policy actions)
consistent	Accepted.
uncertain	Accepted but flagged.
unlocated (no GPS)	Accepted but flagged – except “arrived at destination office”, which is **held**.
mismatch	**Held** for review.
Mock (fake) location reported	**Held** for review.
Capture time more than 5 minutes in the future	**Held** – the phone’s clock may be wrong.

GPS IS EVIDENCE, NOT PROOF

A GPS position only says whether a scan is *consistent* with an approved place. It can never prove that the letter, the postman and the recipient were together. That is why the messages and the tracking page never say “GPS confirmed delivery”, and why a “delivered” scan also needs an acknowledgement.

4.7 Delivery proof and reasons

- **Delivered** needs an acknowledgement: the recipient’s **signature** on the phone screen, the recipient’s **name** (and relationship), or the number of a paper **delivery sheet**. Without one the scan is rejected (`ACK_REQUIRED`), unless the postman writes an approved exception reason – then it is **held** for the postmaster (test T10).
- **Delivery attempted** and **return started** need a reason (e.g. “nobody home”, “address not found”).

4.8 The review queue

A postmaster sees every held scan: the letter, the requested status, the current status, the location result, who scanned it and when, and the reasons. They must type a reason (at least 3 characters) and press **Approve** or **Reject**. Approval applies the status and queues the messages – but only if that move is still valid. If the letter has moved on meanwhile, the system says the event “can no longer be applied; reject it instead”.

PART 5

The phone app – PostAssist Field

For postmen and hub officers: scan the label, record the step, capture the signature – even without network.

5.1 The screens

Screen	What is on it
Sign in	Server address (the laptop running the backend, e.g. `http://172.20.10.5:8000` on the same Wi-Fi), username and password. After the first sign-in the app works offline.
Home	Online/offline state; counts of **pending**, accepted, held and rejected scans; **Scan registered letter**; **Upload now**; the recent scans on this phone with their result; **Sign out** (blocked while scans are still pending, so nothing is lost).
Scan label	The camera reads the Code 128 barcode or the QR code. For a damaged label, type the ID printed under the barcode.
Event	The current status and destination; the recipient's name and address – **only for a postman of the destination office**; the statuses this user may record now; reason buttons for attempts; acknowledgement type (signature pad, recipient name, paper sheet); an optional exception reason; **Save scan (records location now)**.

The location is read once, when Save is pressed, and is shown to the postmaster as evidence quality – not as proof of delivery.

5.2 How offline works

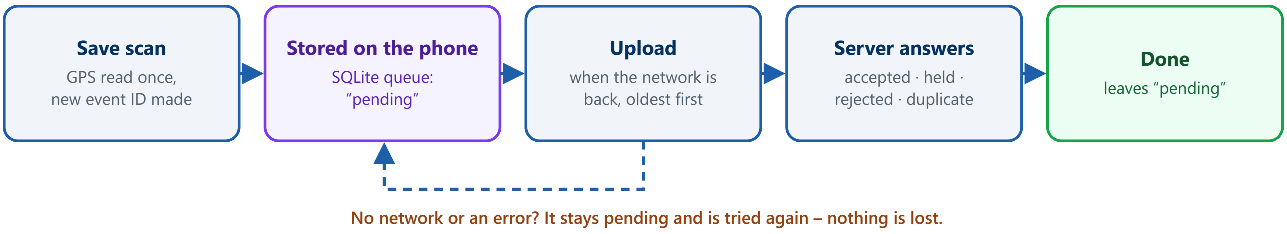

Save scan
GPS read once, new event ID made
Stored on the phone
SQLite queue: "pending"
Upload
when the network is back, oldest first
Server answers
accepted · held · rejected · duplicate
Done
leaves "pending"
No network or an error? It stays pending and is tried again – nothing is lost.

Figure 9 – Every scan is saved on the phone first, then uploaded. The same event ID is used on every try.

1. **Save scan** reads the GPS once and builds the scan with a brand-new event ID.
2. The scan is written to a small database **on the phone** (SQLite) as "pending" **before** any upload is attempted. It survives the app being closed or the phone restarting (test T11).
3. When the network is available – automatically when it comes back, or with **Upload now** – pending scans are sent in the order they were captured, up to 50 at a time.
4. For each scan the server returns a lasting answer: accepted, held, rejected or duplicate.

5. Only then does the scan leave “pending”. A network error keeps it pending to be tried again.

WHY THE SAME EVENT ID MATTERS

Imagine the upload reaches the server but the answer is lost on the way back. The phone sends the scan again. Because it carries the **same event ID**, the server recognises it and answers “duplicate” instead of saving a second scan and sending a second e-mail. An automated test sends the same scan 1,000 times and checks that exactly one event and one message exist (test T12).

PART 6

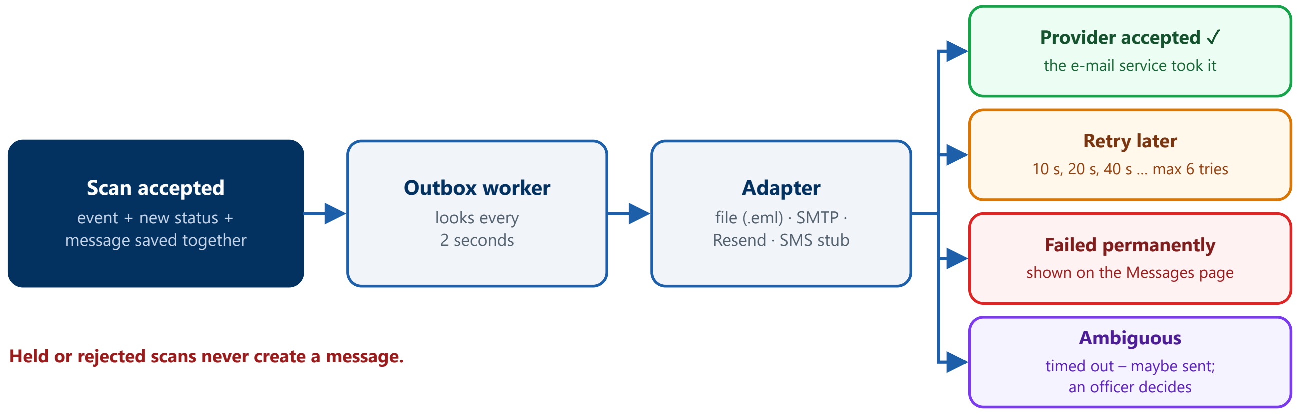
Messages to the sender

How the system makes sure Nimal is told – honestly, once, and in his language.

6.1 When a message is created

- Only for scans that were **accepted** (or accepted after review). Held, rejected and pending scans **never** create a message (tests T10, T14).
- Only if the sender chose e-mail or SMS and gave a contact.
- **Key updates** (default): accepted, out for delivery, delivery attempted, delivered, return started, returned. **All updates** adds dispatched, in transit and arrival.

6.2 The outbox


Scan accepted
event + new status + message saved together
Outbox worker
looks every 2 seconds
Adapter
file (.eml) · SMTP · Resend · SMS stub
Provider accepted ✓
the e-mail service took it
Retry later
10 s, 20 s, 40 s … max 6 tries
Failed permanently
shown on the Messages page
Ambiguous
timed out – maybe sent; an officer decides

Held or rejected scans never create a message.

Figure 10 – The transactional outbox: the message is written together with the scan, then a background worker delivers it.

The message is written into the letter’s own record **in the same database write** as the scan and the new status. So if the scan is not saved, the message does not exist either. A background **outbox worker** inside the backend looks every 2 seconds for messages that are due and sends them through an **adapter**:

Adapter	What it does
file (default)	Writes each e-mail as an `.eml` file in `backend/storage/outbox_mail/`. Free, needs no account – ideal for the demo; open the file to read the e-mail.
smtp	Sends through any e-mail server (e.g. a Mailtrap test inbox or a university mail server).
resend	Sends through the Resend e-mail service, with an idempotency key so it never sends the same e-mail twice.
sms	A placeholder that only logs; a real SMS gateway is optional and not assumed to be free.

Message states

State	Meaning
`queued`	Waiting for the worker.
`sending`	The worker has claimed it (for at most 60 seconds).
`provider_accepted`	The e-mail service accepted it. This is **not** proof that the sender read it.
`retry`	A temporary failure; it will be tried again after 10 s, 20 s, 40 s, 80 s … (never more than 1 hour apart), at most 6 attempts.
`failed_permanent`	It will not succeed (e.g. invalid address, or 6 failed attempts). Shown to the postmaster.
`ambiguous`	We cannot know whether it was sent (e.g. a timeout after the request left, or the worker stopped while sending). It is retried only with a provider that removes duplicates; otherwise an officer decides.
`resolved_manually`	An officer dealt with it (e.g. phoned the sender) and wrote how.

6.3 Three languages, careful wording

Templates exist in English, Sinhala and Tamil (template version `2026.10-draft1`; the Sinhala and Tamil wording should be reviewed by native speakers before real use). Messages never include a precise address or a worker's coordinates, and the "delivered" message says the item was recorded as delivered "under the applicable acknowledgement procedure" – it does not claim GPS proof. If a scan was uploaded more than 10 minutes after it was captured, the message adds "This update was synchronised at …".

EXAMPLE – THE "ACCEPTED" E-MAIL (ENGLISH)

Subject: Registered item PA-DEMO-7K3M9Q2XRH4: Accepted
Body: Your registered item PA-DEMO-7K3M9Q2XRH4 was accepted at SLIIT Malabe demo counter on 18 Oct 2026 10:45. Keep your receipt. View updates through your tracking link: https://…/track/PA-DEMO-7K3M9Q2XRH4#t=…

6.4 The Messages page

For postmasters and counter officers: a filter by state, counts per state, and a table of messages (item, event, channel/language, state with the last attempt's detail, attempts, time). **Preview** shows the exact text. For failed or ambiguous messages: **Retry** (asks why, e.g. "provider back online") and **Resolve** (asks how, e.g. "sender phoned"). Both are audited.

PART 7

The sender’s tracking page

A public page with no login – private by design.

The receipt carries a link and a QR code like:

```
https://localhost:5173/track/PA-DEMO-7K3M9Q2XRH4#t=Zk3pQ…secret…
```

- The part after `#` is a **secret token** made for this letter. Browsers never send the part after `#` to the server as part of the address, so it does not appear in server logs. The page sends it privately in a header instead.
- A wrong token and a tracking ID that does not exist give **exactly the same** “not found” answer. So nobody can discover other people’s letters by changing the ID in the address (test T16).
- The page shows: the tracking ID, when and where it was accepted, the destination **town**, the current status, and a timeline of accepted scans with time and **office name**; late uploads are marked “synchronised later”.
- It never shows GPS coordinates, worker names, the recipient’s name or signature details.
- If a link leaks, a postmaster can **rotate** it: a new token is made and the old link stops working.

PART 8

Supervision and the other web pages

Every page of the web app, who can open it, and what it is for.

The left sidebar shows only the pages your role may use. The search box at the top finds an item by tracking ID from any page.

Page	Who	What it shows and does
Dashboard	All staff	Tiles: registered today, unfinished drafts, on the way, out for delivery, delivered, held for review, failed messages, IoT scale online/offline. Recent activity table, quick actions (start a new registration, record a scan, search, IoT scale, review), a bar chart of intake per hour (08:00–17:00), items by status and sender-message counts. Postmen are told to use the phone app.
Counter intake	Counter officer, postmaster	Part 3.
Items	All staff	Search by tracking ID and filter by status (chips). Columns: ID, status, destination town, weight, accepted time. Hub officers see only routing data (town and postcode), not names.

Page	Who	What it shows and does
Item detail	Staff of the origin or destination office (hubs: any)	Full event history (staff view, with locations and decisions), recipient and sender, weight, charge and tariff, assigned postman, label prints and verification, message states. Postmasters can **assign a postman**. Counter staff can reopen the label/receipt.
Record scan	Counter, hub officer, postmaster	Desk scanning (Part 4.4).
Review queue	Postmaster	Held scans with Approve / Reject (reason required) and the list of "accepted but flagged" scans of the last 14 days.
Messages	Postmaster, counter officer	The outbox (Part 6.4).
IoT scale	Counter officer, postmaster	Live device status, tare, calibrate, test photo, weighing trials (Part 9.8).
Offices & zones	Postmaster (own office)	Each office's reference point (latitude, longitude), facility radius and delivery-area radius. **Use my current location** fills the position from the browser. A **provenance** note (where the numbers came from) is mandatory. A link opens the point on OpenStreetMap.
Reports	Postmaster (switching: admin)	Downloads of three CSV files without names or addresses (intake tasks, scan events, notifications); the active location policy and – for the administrator – switching between L0, L1 and L2 with a reason; tariff versions and "Mark approved".
Audit	Postmaster	Search the audit log by action prefix (e.g. `scan.`), target (tracking ID or draft) and severity (security, warning, info).
Label print	Counter officer	Part 3.11.
Track	Public	Part 7.

The web app is designed to be usable with the keyboard alone: a "Skip to content" link, a visible focus ring, labelled fields, and badges that always use **text and colour** (never colour alone).

PART 9

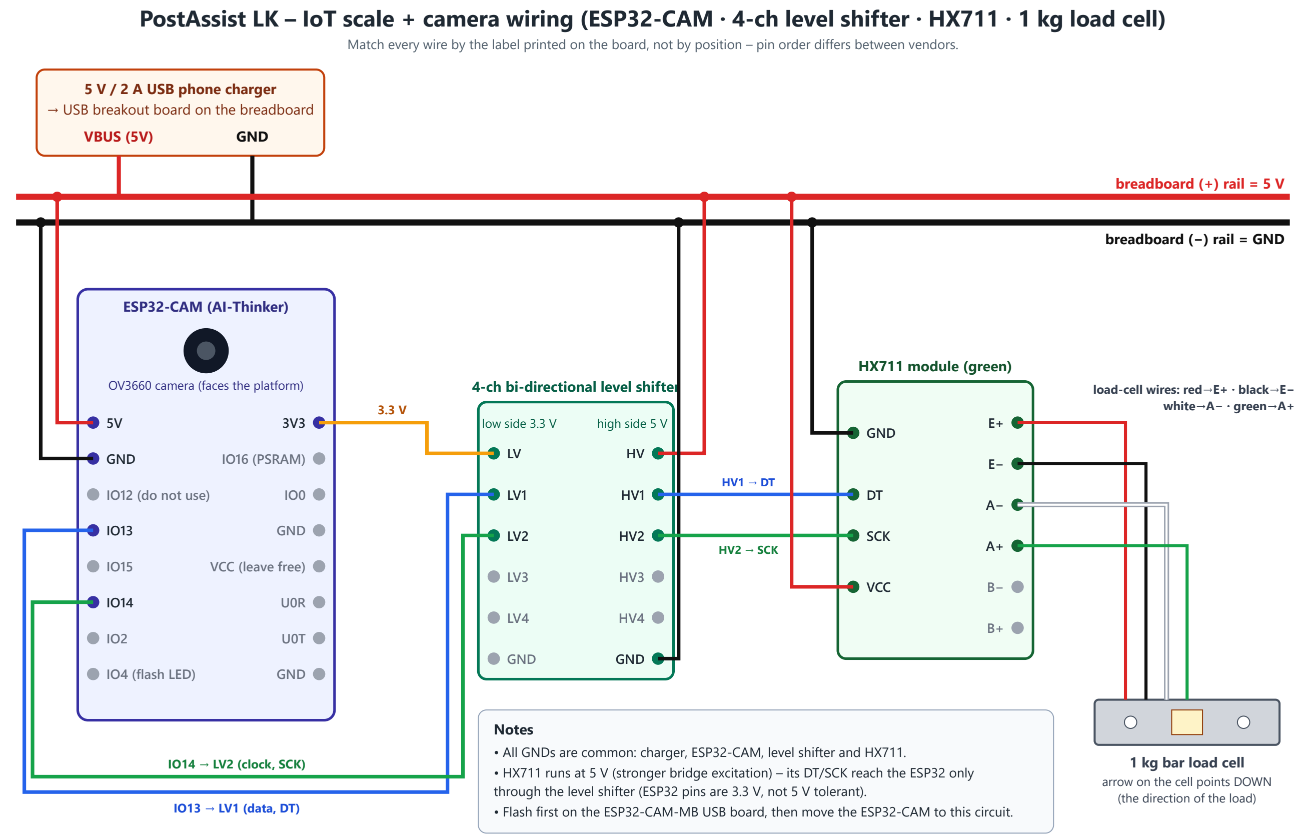
The IoT scale and camera

The hardware you built: what each part does, how it weighs, and how it talks to the backend.

9.1 The parts

Part	What it does	Cost
ESP32-CAM (AI-Thinker) with OV3660 3 MP camera	The brain: a tiny Wi-Fi computer with a camera. Runs the firmware, reads the scale, takes photos and talks to the backend.	Rs 2,750
1 kg bar load cell	A metal bar with strain gauges. It bends a tiny amount under weight and changes a small electrical signal.	Rs 450
HX711 amplifier	Measures that tiny signal very precisely (24-bit) and turns it into numbers, 10 per second.	Rs 160
Wooden bracket / platform	Holds the load cell: one end fixed, the other carries the platform.	Rs 600
4-channel level shifter	Translates between 5 V (HX711) and 3.3 V (ESP32), so no 5 V signal damages the ESP32.	Rs 100
Jumper wires, shipping	Connections.	Rs 740
5 V / 2 A USB charger, breakout, breadboard	Power for everything.	–

Purchased total Rs 4,800. Wiring: HX711 DT → GPIO 13 and SCK → GPIO 14 (through the level shifter); GPIO 12 and 16 must not be used. The camera is mounted 28–32 cm above the platform, lens down, with a desk lamp for even light.



PostAssist LK – IoT scale + camera wiring (ESP32-CAM · 4-ch level shifter · HX711 · 1 kg load cell)

Match every wire by the label printed on the board, not by position – pin order differs between vendors.

5 V / 2 A USB phone charger
→ USB breakout board on the breadboard
VBUS (5V)
GND
breadboard (+) rail = 5 V
breadboard (−) rail = GND
ESP32-CAM (AI-Thinker)
OV3660 camera (faces the platform)
5V
3V3
GND
IO16 (PSRAM)
IO12 (do not use)
IO0
IO13
GND
IO15
VCC (leave free)
IO14
U0R
IO2
U0T
IO4 (flash LED)
GND
3.3 V
4-ch bi-directional level shifter
low side 3.3 V
high side 5 V
LV
HV
LV1
HV1
LV2
HV2
LV3
HV3
LV4
HV4
GND
GND
HV1 → DT
HV2 → SCK
HX711 module (green)
GND
E+
E−
DT
A−
SCK
A+
VCC
B−
B+
load-cell wires: red→E+ · black→E−
white→A− · green→A+
1 kg bar load cell
arrow on the cell points DOWN
(the direction of the load)
IO14 → LV2 (clock, SCK)
IO13 → LV1 (data, DT)

Notes

- All GNDs are common: charger, ESP32-CAM, level shifter and HX711.
- HX711 runs at 5 V (stronger bridge excitation) – its DT/SCK reach the ESP32 only through the level shifter (ESP32 pins are 3.3 V, not 5 V tolerant).
- Flash first on the ESP32-CAM-MB USB board, then move the ESP32-CAM to this circuit.

Figure 11 – Wiring of the ESP32-CAM, level shifter, HX711 and load cell (from Documents/IoT_Hardware_Setup_Guide.md).

9.2 How weighing works

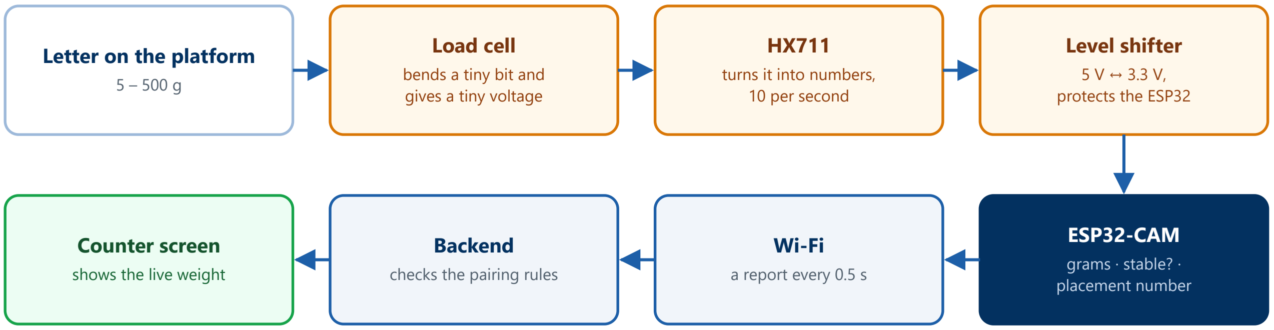

Letter on the platform
5 – 500 g
Load cell
bends a tiny bit and gives a tiny voltage
HX711
turns it into numbers, 10 per second
Level shifter
5 V ↔ 3.3 V, protects the ESP32
ESP32-CAM
grams · stable? · placement number
Wi-Fi
a report every 0.5 s
Backend
checks the pairing rules
Counter screen
shows the live weight

Figure 12 – From a letter on the platform to the number on the counter screen.

- The firmware keeps the last **10 readings** (one second). The **spread** is the largest minus the smallest of them. If the spread is at most **1 g**, the reading is **stable**.
- Above **2 g** the platform counts as **loaded**; below **1 g** (and stable) as **empty** – the next load then gets a new **placement number**.
- While empty, slow drift of up to 0.5 g is quietly re-zeroed.
- A single reading that jumps more than 25 g away from the others is ignored unless the next reading confirms it (a real letter is confirmed at once; an electrical glitch is not).

9.3 Tare and calibrate

- **Tare** (platform empty): “the current reading is zero”.
- **Calibrate**: after taring, put a known reference mass (10–1000 g, e.g. 100 g) on the platform and press Calibrate with that value. The scale works out its **factor** (raw counts per gram), creates a new **calibration ID**, and stores both in its flash memory, so they survive power cuts.
- Because the zero point and the factor are both saved in flash, the scale keeps working after a restart – no new tare is needed. (An older “pull” board connected through the bridge loses its zero when it restarts; the system then shows “needs tare” and refuses its weight until it is zeroed again.)
- Safety (firmware 1.1.1): a factor that is impossibly small (below 100 counts/g) is refused, and there are reset options, so one bad calibration can no longer lock the scale.
- A new calibration clears the old “±” uncertainty until new weighing trials are done.

9.4 How the scale talks to the backend

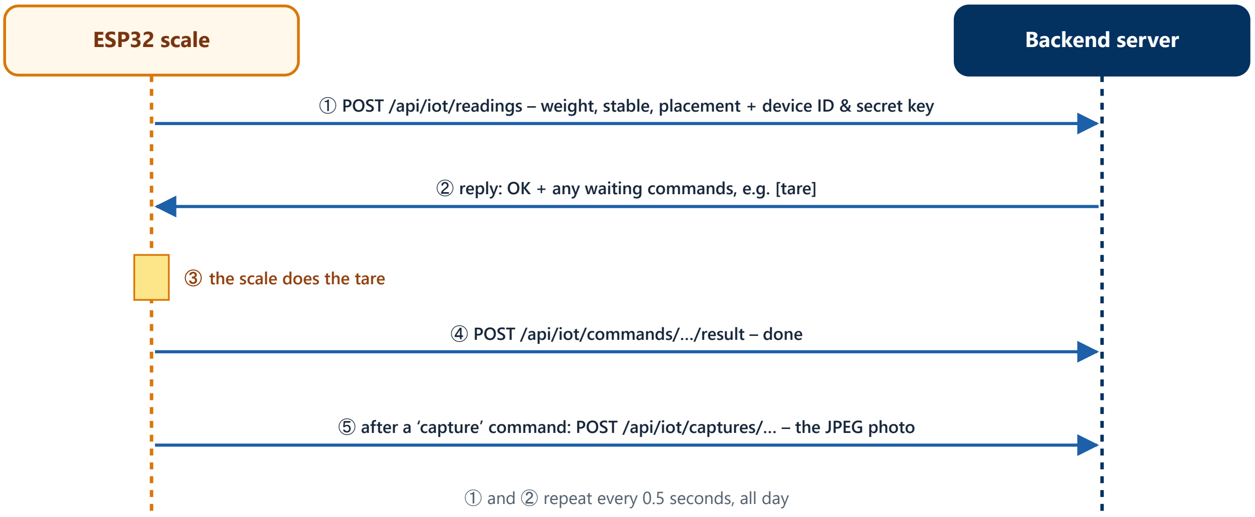

ESP32 scale
Backend server
① POST /api/iot/readings – weight, stable, placement + device ID & secret key
② reply: OK + any waiting commands, e.g. [tare]
③ the scale does the tare
④ POST /api/iot/commands/.../result – done
⑤ after a ‘capture’ command: POST /api/iot/captures/... – the JPEG photo
① and ② repeat every 0.5 seconds, all day

Figure 13 – The conversation between the scale and the backend. Commands ride back on the answer to each reading, so the scale needs no open port.

- Every request carries the headers `X-Device-Id` and `X-Device-Key`. The backend stores only a fingerprint (SHA-256) of the key; a wrong key is refused and logged as a security event.
- Each answer carries up to 3 waiting commands. A command not picked up within 60 seconds expires.
- If no reading arrives for 10 seconds, the device is shown as **offline**.
- Readings are stored for diagnosis (always when loaded or when something changes, otherwise every 30 s) and **deleted automatically after 3 days**. A weight that is paired with a letter is copied into that letter, so nothing important is lost.

9.5 The camera

Photos are 2048 × 1536 pixels (QXGA) JPEG at quality 10, with lens correction and edge sharpening (+2). Three frames are thrown away first so exposure and colour settle, then a burst of three is taken and the frame with the **most detail** (the largest JPEG) is sent. The image is flipped because the module is mounted upside down. The **focus helper** on the device page shows a detail score while you turn the lens: stop where it peaks.

9.6 The device's own web page

Open `http://<device-IP>/` in a browser on the same network: live weight, Tare, Calibrate, test photo, focus helper, and settings for the Wi-Fi name, password and backend address (no re-flashing needed). If the scale cannot join any Wi-Fi it creates its own set-up network **PostAssist-Scale** at `http://192.168.4.1/`.

9.7 The settings file

`iot/postassist_scale_cam/config.h` holds the Wi-Fi name and password, the backend address (e.g. `http://172.20.10.5:8000`), the device ID (`SCALECAM-01`) and the device key. It contains secrets, so it is **not** stored in git; `config.example.h` is the template. Board settings in the Arduino IDE: "AI Thinker ESP32-CAM", partition "Huge APP", PSRAM enabled, library ArduinoJson 7.

9.8 The IoT scale page in the web app

- Header: device ID, **online** / **offline**, IP address, firmware version, Wi-Fi signal strength (RSSI) and last-seen time.
- A large live weight with badges: stable / settling / needs tare, placement number or empty, HX711 ok / fault, camera ok.
- **Zero and calibrate**: Tare, and Calibrate with a reference mass. The result appears (e.g. "calibrated: CAL-… – reads 100.0 g with the reference mass on it").
- **Camera test**: take a test photo and see its size and quality result.
- **Weighing trials**: enter the reference mass and condition (centre, off-centre, re-placed, after restart, near a band boundary) and record the current stable reading. The page shows the number of trials and the **MAE** (mean absolute error) and downloads a CSV.
- Postmasters enter the **expanded uncertainty U** calculated from the trials by `ml/evaluation/weighing_analysis.py`, with the evidence.

9.9 Without the hardware

`backend/tools/device_simulator.py` pretends to be the scale and camera, so the whole counter flow can be shown without the board. `backend/tools/device_bridge.py` connects an older board that only answers questions (a "pull" sketch); it is not needed with firmware 1.1, which reports by itself.

PART 10

The “smart” parts (AI)

Three trained models, one free OCR engine and one optional language model – what each does and how well.

WHAT A MODEL IS

A child learns to read by seeing thousands of letters, not by memorising rules. A machine-learning model is the same: it is shown many examples with the right answers (**training**), adjusts itself until it gets them right, and is then tested on examples it has **never seen**. Only that last test tells you how good it really is. The trained result is a file (here, in the `backend/models/` folder) that the backend loads.

10.1 Script identification (CNN)

- **Task:** look at one line of text and say **Sinhala, Tamil or English**, so Tesseract can use the right language pack (faster and more accurate than trying all three).
- **How:** a small *convolutional neural network* (about 0.46 million settings) that looks for the shapes typical of each script.
- **Data:** lines of public text drawn in 18 fonts for training, 4 for validation and **6 fonts never seen** for testing.
- **Result: 98.6 %** correct on the unseen fonts; 2–7 milliseconds per line on an ordinary CPU. Weak spot: handwriting-style English (87.5 % ± 7.9 % across training runs), so P1 always adds English for mixed text and the officer can choose the script.
- Its confidence is “temperature-scaled” so that 90 % confident really means right about 90 % of the time.

10.2 The handwriting reader (CRNN + CTC)

- **Task:** read one handwritten line – Sinhala, English or Tamil, or mixed – into characters. Tesseract is built for print and fails on handwriting.
- **How:** 7 convolution layers look at the line from left to right, 2 bidirectional LSTM layers read the sequence in both directions, and **CTC** decoding lines the guesses up into text. 4.5 million settings, 195 characters, about 13 ms per Sinhala line.
- **Before the network:** the photo becomes an “ink image” (paper = 0, pen = 1), the block is straightened and each line is cut out by its writing.
- **Training data:** 820 real Sinhala lines (SinOCR handwritten), 6,482 real English lines (IAM), synthetic Sinhala address lines spelled with real handwritten characters (a 454-class character set), and fonts for English, Tamil and missing Sinhala letters. **No real Tamil handwriting** was available.

Test set (held out)	Lines	Model CER	Model WER	Tesseract CER
Real Sinhala handwriting (SinOCR test)	227	**18.1 %**	55.5 %	91.1 %
Real English handwriting (IAM test sample)	400	**9.2 %**	31.1 %	55.4 %

Test set (held out)	Lines	Model CER	Model WER	Tesseract CER
Tamil in fonts never seen (synthetic)	300	**0.4 %**	2.4 %	–
English address text, unseen handwriting-style font	300	**2.0 %**	13.1 %	–

HOW TO READ THESE NUMBERS HONESTLY

- A CER of 18 % means roughly one character in five or six is wrong; a WER near 50 % means about every second handwritten word still has a mistake. The model **saves typing** – it does not replace reading the envelope.
- On your own two test photos the error was about 7 % (English) and 21 % (Sinhala).
- **Tamil handwriting is untested** – the Tamil row only shows that it reads Tamil fonts.
- House numbers and postcodes must always be checked: inside a number the decoder prefers digits (l → 1, O → 0) and keeps the word flagged.

10.3 Tesseract

A free OCR engine (version 5.5) for **printed** text, used with the `sin`, `tam` and `eng` language packs. If it is not installed, the counter still works: the badge says so and the officer types or pastes the address.

10.4 The address parser (CRF)

- **Task:** label every word of the address text as NAME, HOUSE, STREET, LOCALITY, CITY, REGION, POSTCODE, COUNTRY or other, and so fill the fields.
- **How:** a *Conditional Random Field* – a classic, explainable model that looks at clues for each word and its neighbours: Is it a number? Is it a known town or district? Is it a word like “Road”, “පාර” or “வீதி”? Is it an honorific like “Mr” or “මයා”? Which line of the envelope is it on?
- **Result: 99.4 %** of addresses from towns **never seen in training** had all required fields exactly right (real OpenStreetMap-based addresses 97.6 %), against only 57.5 % for simple hand-written rules.
- **Confidence and threshold:** each field gets a confidence; the review threshold (0.974) was chosen on validation data and then frozen. It flags only 1.7 % of fields yet catches 55 % of the errors; un-flagged fields are 99.7 % correct.

10.5 Optional LLM tidy-up and the guard

An optional local language model (Gemma 3 4B through Ollama – switched off by default) can tidy an address. Whatever produced a suggestion, a **guard** compares it with the original text and refuses any **unsupported change**: a digit that was not there (an invented house number or postcode), a name word that was not there, or a change of script. The officer then sees the original value and the reason.

10.6 Fallbacks

If a model file is missing, the system falls back – to simple rules for the parser, to Tesseract only for reading, or to manual typing – so the counter is **never blocked**.

PART 11

Safety, privacy and reliability

The rules that make the system trustworthy, and the reason for each.

What the system does	Why
Stores passwords only as **PBKDF2-SHA256 hashes** (390,000 rounds, random salt)	A stolen database does not reveal passwords.
Signed login tokens that **expire after 12 hours**; the user is re-read on every request	A deactivated account stops working immediately.
Role rules on every endpoint and **office / assignment rules** on every letter	An officer of another office cannot see or scan your letters (OWASP API security risk no. 1). Every refusal is logged as a security event (test T07).
The worker's identity comes from the login token, never from the request	Nobody can record a scan in someone else's name.
The scale has its own key; only its fingerprint is stored	A fake device cannot send weights.
No personal data in barcodes, QR codes, URLs, the audit log or CSV exports	Protects senders and recipients (Personal Data Protection Act No. 9 of 2022).
Tracking secret after the `#`; wrong token = unknown ID	Links cannot be guessed and do not leak into logs.
No double records: one item per draft; one letter per scale placement; one event per event ID; one message per event + recipient + channel + template	Retries, double clicks and replays can never create duplicates.
Atomic saving: event, status and messages in one database write, guarded by the revision number	No message for a scan that was never saved; no lost updates when two people act at once.
Audit log of logins, refusals, acceptances, scans, reviews, reprints, manual weights, policy and tariff changes	Every important action can be explained later: who, what, when, why.
Envelope photos in `backend/storage/images/`, excluded from git, visible only to staff of the same office	Photos contain personal data.

What the system does	Why
Versioned tariffs and location policies; every quote and scan stores the version used	Every price and every decision can be reproduced and explained.

PART 12

Where the data is kept

The MongoDB database `postassist_registered_mail` and a few folders on disk.

Collection	What is in it	Personal data · how long
`users`	Staff accounts: username, full name, role, office, delivery beat, password hash.	Staff names · project duration
`offices`	The six demo offices with reference point, zone radii and provenance (most from OpenStreetMap).	None
`drafts`	Intakes in progress: fields, OCR runs, suggestions, corrections, timings, weight, quote, status (draft / committed / cancelled).	Sender, recipient · delete after evaluation
`items`	Accepted letters, each holding its **events** and its **outbox** inside it, plus label prints, payment and the public token.	Sender, recipient, contact, signature · per approved protocol
`images`	Information about each envelope photo (size, quality result, file path).	Yes (the photo) · per protocol
`devices`	The scale: office, key fingerprint, latest reading, calibration, IP, firmware.	None
`device_readings`	Raw scale readings for diagnosis.	None · auto-deleted after 3 days
`device_commands` · `captures`	Commands sent to the scale and photo requests, with results.	None
`weighing_trials`	Reference-mass trials for the weighing experiment.	None
`tariffs` · `location_policies`	Versioned price lists and location rules. Old versions are never edited.	None · permanent
`audit_log`	Actor, action, target ID, details, severity, time.	Staff usernames · project duration

Files on disk: `backend/storage/images/` (photos), `backend/storage/outbox_mail/` (e-mails from the file adapter) and, if no key was configured, `backend/storage/seed_device_key.txt`. The database files themselves are in `%USERPROFILE%\PostAssist\mongo-data`.

IN SIMPLE WORDS – WHY EVENTS LIVE INSIDE THE ITEM

Each letter is one document that contains its whole history and its messages. Changing a letter is therefore always **one** write to **one** document, which MongoDB guarantees to be all-or-nothing. That is how the system saves a scan, the new status and the sender's message together without needing complicated multi-document transactions.

Starting data (seed): on first start the backend creates – only if missing – the six offices, one account per role, the 2026 tariff, the default L2 location policy and the scale `SCALECAM-01`. Backups: `mongodump` / `mongorestore`.

PART 13

How to start and use the system

A short practical guide. The full version is `Documents/Setup_and_Run_Guide.md`.

13.1 The easy way

1. Double-click `backend\tools\start_all.bat`. It starts MongoDB if it is not already running, then opens one window for the API (backend) and one for the web app. (The backend would also start MongoDB by itself.)
2. Open **https://localhost:5173** in Chrome or Edge. The first time, the browser warns about the security certificate (it is a local, self-made one) – choose to continue.
3. Log in, e.g. as `counter2`.
4. Switch the scale on. It joins the "Senomie's iPhone" hotspot and starts reporting to `http://172.20.10.5:8000`. The IoT scale page should show **online**.

13.2 Manually (PowerShell, from the project folder)

```
cd backend
.venv\Scripts\python -m uvicorn app.main:app --host 0.0.0.0 --port 8000

cd frontend
npm run dev
```

13.3 Addresses and ports

Address	What it is
`https://localhost:5173`	The web app (counter, items, review ...).
`http://localhost:8000/docs`	The backend's automatic API documentation – every endpoint can be tried here.

Address	What it is
`http://localhost:8000/api/health`	Health check: answers “ok” when the backend and database work.
`127.0.0.1:27017`	MongoDB (only this computer can connect).
`http://<scale-IP>/`	The scale’s own page.
`http://172.20.10.5:8000`	The laptop on the iPhone hotspot – the address the scale and phones use.

13.4 Logins

All demo accounts use the password set as `DEMO_PASSWORD` in `backend/.env`.

Username	Role	Office
`counter2` · `hubdemo` · `postman2` · `pm2`	counter · hub · postman · postmaster	SLIIT Malabe demo counter (the scale’s office)
`counter1`	counter officer	Kaduwela Post Office
`hub1`	hub officer	Central Mail Exchange, Colombo
`hubjaf` · `postman1` · `pm1`	hub · postman (beat 40000) · postmaster	Jaffna Main Post Office
`admin`	administrator	General Post Office, Colombo

13.5 Settings file (`backend/.env`)

Holds the database address, the JWT secret, the demo password, the scale’s key, the e-mail adapter and the web address used in tracking links. It contains secrets and is **not** stored in git (`.env.example` is the template). Never share it.

13.6 Automated tests

```
cd backend
.venv\Scripts\python -m pytest -q
```

More than 70 tests check the rules automatically (weight pairing, tariff bands and the boundary rule, tracking-ID typos, the full journey with location review and messages, 1,000 replays, provider failures, handwriting pre-processing, the device bridge …). They need MongoDB running.

PART 14

The research side

What the component is meant to prove, how it is measured, and what is still open.

14.1 The four objectives

Objective	Target (from the proposal)	How it is measured
SO1 Assisted capture	At least 20 % lower median intake time than manual entry; at least 98 % confirmed required-field exact match; handwriting failures reported.	B0/B1/P1 configurations, `intake_tasks.csv`, OCR benchmark, usability sessions.
SO2 IoT weighing and tariff	Mean absolute error ≤ 1 g over 5–500 g; 100 % tariff agreement including band edges; boundary uncertainty never silently rounded.	Weighing trials → `weighing_analysis.py`; tariff tests.
SO3 Location-aware scans	Compare L0/L1/L2; at most 5 % false acceptance of clearly-outside scans.	Policy switch, `scan_events.csv`, field trials; simulation of 20,000 scans per scenario.
SO4 Offline sync and messaging	Zero duplicate events in 1,000 replays; zero messages from rolled-back or pending events; all saved offline events recovered.	Automated replay and outbox tests; airplane-mode test on a phone.

SIMULATION RESULT FOR SO3 (20,000 SCANS PER SCENARIO)

False acceptance of clearly-outside scans: L0 100 %, L1 0 %, **L2 0 %**. But L1 turns 6.9 % of valid scans into hard mismatches and accepts 90 % of stale GPS fixes, while L2 makes only 1.7 % mismatches and accepts 0 % of stale fixes – at the cost of flagging 18.7 % of valid scans as uncertain.

14.2 Acceptance tests

Test	Situation	Expected	Status
T01	Clear printed envelope	Officer confirms; suggestion and final value both kept	implemented
T02	Blurred handwritten Sinhala/Tamil	Recapture or manual entry; nothing invented	implemented
T03	Previous letter's weight offered to a new draft	Refused; fresh reading needed	pass
T04	Weight uncertainty overlaps a band boundary	Reweigh / approved exception	pass
T05	Accept repeated after a timeout	Same item and receipt	pass
T06	Printer fails after acceptance	Reprint with same ID, audited	pass
T07	Unassigned worker scans a valid code	Denied, security audit	pass

Test	Situation	Expected	Status
T08	GPS clearly outside the facility	Held; no misleading arrival	pass
T09	Location denied or stale	Unlocated / uncertain by policy	pass
T10	Delivered without acknowledgement	Rejected/held; no delivered message	pass
T11	Offline save, then app restart	Pending survives with the same ID	run on a phone
T12	Same scan uploaded 1,000 times	One event, one message	pass
T13	Late transit scan after delivered	History only	pass
T14	Held / rolled-back event	No message	pass
T15	Provider timeout after possible acceptance	Ambiguous handled honestly	pass
T16	Sender changes the ID in the URL	Nothing disclosed	pass
T17	Approved return after failed attempts	Return states, attempts kept	pass
T18	Map / geocoder unavailable	Counter and timeline continue	pass by design

14.3 Honest limitations

- No real customer mail yet – synthetic mail and role-played participants until ethics and Department of Posts approval.
- Sinhala and Tamil training text is largely synthetic; **Tamil handwriting is untested**; handwriting results on new writers at the counter will be worse than on the test sets.
- The DIY scale is a research demonstrator, not a legal-for-trade instrument; official charging uses an approved scale. The HX711 wiring needs care (level shifter, slow clock).
- The tariff is transcribed by the student and must be approved by an authorised officer.
- GPS is uncertain supporting evidence, never proof.
- Tracking IDs use the `PA-DEMO` prefix until an official numbering range is allocated.
- Not built: recipient OTP, push notifications, a map view of scans.

PART 15

The teammate’s Postal KPI desk

Another group member’s component that shares the same server – it is **not** part of this component.

When the branches were merged, the teammate’s **Postal KPI desk** (employees, daily KPI log, attendance with QR, productivity forecast, entitlements and payroll) was combined with this component so that **one backend on port 8000 serves both web apps**.

- Its API (`backend/routes.py` , `db.py` , `attendance.py` , `payroll.py` , `productivity.py` , `excel_io.py`) is attached at the **end** of `backend/app/main.py` , after every registered-mail route, so its general paths can never hide ours. A test checks this order.
- If its extra Python packages are missing, the registered-mail system still starts and only logs a warning.
- Its pages (`frontend/src/admin` , `attendance` , `entitlements`) use Material UI and live under their own routes such as `/employee` , `/daily-kpi-log` and `/entitlements` .
- It has its own health check at `/health` ; ours is `/api/health` .

APPENDIX A

Every file, explained

Where each piece of the component lives in the repository.

A.1 Backend – start-up and shared code (`backend/app/`)

File	What it does
`main.py`	Creates the app; at start-up starts MongoDB if needed, creates indexes, adds the demo data and starts the outbox worker; attaches all routers; `/api/health` ; attaches the KPI desk last.
`config.py`	Reads all settings from `backend/.env` (database, secrets, OCR, weighing limits, e-mail, demo data, MongoDB auto-start).
`db.py`	Connects to MongoDB, starts a local MongoDB when none is running, and creates the indexes that enforce the “no double records” rules.
`security.py`	Password hashing, login tokens, role checks (`require_roles`) and device-key checks.
`audit.py`	Writes one audit-log entry; a failure to audit never breaks the request.
`schemas.py`	The exact shape of every request (fields, types, limits), checked automatically.
`seed.py`	Demo offices, users, tariff, location policy and the scale device.

File	What it does
`timeutil.py`	Times in UTC, shown in Sri Lanka time.
`reference/`	Copies of the gazetteer, post-office list, trilingual vocabulary and the 2026 tariff.

A.2 Backend – routers (the API doors, `backend/app/routers/`)

File	What it does
`auth.py`	Log in; “who am I”.
`intake.py`	The counter: drafts (create, list, resume, update, cancel), images, scale capture, OCR, parsing, address check, confirm fields, weight pairing and manual weight, live weight, quote, accept.
`items.py`	Item list and detail, label data, label printed / reprint, label verify, assign postman, recipient pin, rotate tracking link.
`scans.py`	Record one scan, upload a batch from the phone, look up an item for scanning, list statuses for a role.
`review.py`	Held and flagged scans, approve/reject, audit-log search.
`notifications.py`	The outbox list, preview, retry and resolve.
`devices.py`	Staff view of the scale: list, live, readings, commands, trials, CSV, uncertainty, new device, new key.
`iot.py`	The doors the scale itself uses: readings, photo upload, command results.
`public.py`	The public tracking API (needs the secret token).
`admin.py`	Offices and zones, tariffs and approval, location policy and switching, users.
`reports.py`	CSV exports (intake, scan events, notifications) and the dashboard summary.
`common.py`	Small helpers: “not found”, coded problems, same-office check.

A.3 Backend – services (the rules, `backend/app/services/`)

File	What it does
`weighing.py`	The pairing rules for scale readings, placement keys and the uncertainty used for the boundary rule.
`tariff.py`	Validates tariffs, selects the one in force, computes itemised quotes and the boundary rule.
`tracking_id.py`	Makes random tracking IDs with a check character; cleans typed IDs; validates them.
`labels.py`	Draws the Code 128 barcode and QR codes as images (SVG).
`state_machine.py`	The nine statuses, allowed moves, late-history rule, which role may record what, which statuses notify.

File	What it does
`scan_events.py`	The eight-check decision pipeline for scans, reference areas, permission rules, review decisions.
`location.py`	Policies L0/L1/L2: distance, probability inside the area, results and actions.
`item_views.py`	What staff see versus the redacted public timeline.
`images.py`	Saves envelope photos to disk and records their quality.
`ocr/quality.py`	Image-quality check (resolution, sharpness, brightness, contrast, glare).
`ocr/pipeline.py`	B1 and P1 reading: crop/rotate, straighten, find lines, script ID, Tesseract, handwriting, choosing the reader, flags.
`ocr/script_id.py`	Loads and runs the script-identification model.
`ocr/htr.py`	Handwriting reader: ink image, straightening, line finding, running the model, decoding text.
`address/text.py`	Unicode clean-up, script detection per character, splitting into words.
`address/lexicon.py`	Gazetteer and vocabulary look-ups; town ↔ postcode check; Colombo zone postcodes.
`address/features.py`	The clues the CRF looks at for every word.
`address/parser.py`	Runs the CRF (or the rule fallback), builds fields with confidence and review flags.
`address/normalizer.py`	Rule tidy-up, optional LLM tidy-up and the guard against invented content.
`notifications/templates.py`	Message texts in English, Sinhala and Tamil.
`notifications/adapters.py`	File, SMTP, Resend and SMS-stub senders with honest outcomes.
`notifications/outbox.py`	Builds outbox entries and runs the background worker (claims, retries, leases).

A.4 Backend – models, tools and tests

File	What it does
`models/address_crf.*`	The trained address parser and its information file.
`models/script_id.*`	The trained script-identification model (ONNX) and its information.
`models/htr.onnx · htr.meta.json`	The trained handwriting reader and its character set and test results.
`tools/start_all.bat · .ps1`	Start MongoDB, the API, the bridge (if needed) and the web app in separate windows.
`tools/device_simulator.py`	A software stand-in for the scale and camera.
`tools/device_bridge.py`	Connects an older “pull” board to the backend.
`tests/test_units.py`	Unit tests: tariff, tracking IDs, location, address parsing, guard.
`tests/test_workflow.py`	End-to-end tests T03–T17 through the real API.

File	What it does
`tests/test_handwriting.py · test_device_bridge.py · test_combined_api.py`	Handwriting pre-processing, the bridge, and the shared server with the KPI desk.
`requirements.txt · .env.example`	The Python packages (exact versions) and the settings template.

A.5 Web app (`frontend/src/`)

File	What it does
`main.jsx · App.jsx`	Starts the app and defines every page address (route), protected by login.
`api.js`	Sends requests to the backend with the login token; turns errors into readable messages.
`auth.jsx`	Remembers who is logged in; `can(user, role…)` decides what to show.
`index.css`	Colours and fonts (navy theme, Sinhala and Tamil font fall-backs).
`components/Layout.jsx`	Navy sidebar, top search bar, account menu.
`components/ui.jsx · icons.jsx`	Buttons, cards, badges, alerts, fields, tiles; the icon set.
`components/ImagePanel.jsx`	Step 1: capture/webcam/upload, crop box, rotate, reading options, OCR summary.
`components/AddressBlock.jsx`	Step 2: address fields with suggestions, confidence and the town/postcode check.
`components/WeightPanel.jsx`	Step 4: live weight, “Use this weight”, manual weight.
`components/StatusBadge.jsx · Timeline.jsx`	Status, location and decision badges; the event history list.
`pages/Login · Dashboard · CounterHome · IntakeWizard`	Sign in; overview; counter page with drafts; the intake screen (steps 3 and 5 live here).
`pages/LabelPrint · Items · ItemDetail · ScanEvent`	Label and receipt; item search; item history; desk scanning.
`pages/ReviewQueue · Notifications · Devices`	Held scans; the outbox; the IoT scale page.
`pages/Offices · Reports · Audit · Track`	Zones; exports, policy and tariffs; audit search; the public tracking page.
`vite.config.js`	Runs the web app on port 5173 with HTTPS and forwards `/api` to the backend.

A.6 Phone app (`mobile_app/`)

File	What it does
`App.js`	Switches between the screens and keeps the session.
`src/screens/LoginScreen.js`	Server address, username, password.

File	What it does
`src/screens/HomeScreen.js`	Counts, upload now, recent scans, sign out.
`src/screens/ScanScreen.js`	Camera barcode/QR reader and typed-ID fallback.
`src/screens/EventScreen.js`	Choose the status, reason, acknowledgement and signature; save.
`src/db.js · sync.js`	The offline queue on the phone, and uploading it in order.
`src/location.js · api.js · components.js`	GPS with accuracy, age and mock flag; requests with timeouts; buttons and styles.

A.7 Device, AI, data and documents

File / folder	What it does
`iot/postassist_scale_cam/*.ino`	The scale firmware (version 1.1.x).
`iot/…/device_page.h · camera_pins.h · config.example.h`	The device’s own web page; camera wiring; settings template.
`ml/address_parser/`	Builds the address training data and trains the CRF.
`ml/script_id/`	Renders text lines in many fonts and trains the script model.
`ml/htr/`	Prepares the handwriting data, trains and evaluates the handwriting reader.
`ml/evaluation/`	OCR benchmark, weighing analysis, location-policy simulation.
`ml/synthetic_envelopes/ · datasets/ · common/`	Synthetic envelope images; downloading and building reference data; shared helpers.
`ml/reports/`	Training logs and results reports for every model.
`Datasets/`	Curated vocabulary and tariff, processed gazetteer and training files, templates for the experiments (weighing, location, usability, envelopes).
`Documents/`	Requirements specification, setup guide, IoT hardware guide, IoT test flow, presentation, folder guide and this guide.

APPENDIX B

Messages you may see, and what to do

The coded messages the system shows, in plain words.

B.1 Weight

Code	Meaning · what to do
`NO_READING`	The scale has not reported yet. Check that it is on and connected to the hotspot.
`STALE_READING`	The last reading is more than 3 s old. Check the scale’s Wi-Fi.
`SENSOR_FAULT`	The HX711 is not answering. Check its wiring.
`NEEDS_TARE`	The scale restarted. Empty the platform and press Tare on the IoT scale page.
`UNCALIBRATED`	Calibrate with a reference mass.
`PLATFORM_EMPTY` · `UNSTABLE` · `NEGATIVE`	Nothing on the platform · wait for it to settle · empty the platform and tare.
`PLACEMENT_ALREADY_USED`	This reading belongs to another letter. Lift the letter off and place it again.

B.2 Price

Code	Meaning · what to do
`NO_WEIGHT`	Pair a scale reading or enter a manual weight first.
`BOUNDARY_UNCERTAIN` (reweigh required)	The weight ± uncertainty crosses a price step. Reweigh on the approved scale or use the exception procedure.
`BELOW_RANGE` · `ABOVE_PROTOTYPE_RANGE`	Below 5 g · above 500 g – use an approved scale and a manual weight.
`INSURED_VALUE_OUT_OF_RANGE`	Declared value must be 1–25,000 LKR.
`TARIFF_NONE_IN_FORCE` · `TARIFF_AMBIGUOUS`	No tariff, or more than one, is valid today – an administrator must fix the tariffs.
`TARIFF_BANDS_NOT_CONTIGUOUS` and other `TARIFF_…`	The tariff data has a gap, overlap or missing fee – quoting is blocked until it is corrected.

B.3 Intake and labels

Code	Meaning · what to do
`FIELDS_INCOMPLETE`	Required fields are missing – the list says which.
`CANNOT_ACCEPT`	Something is not ready (not confirmed, no weight, quote expired, payment different) – the list says what.
`DRAFT_CLOSED`	The draft was already accepted or cancelled.

Code	Meaning · what to do
`NO_IMAGE` · `IMAGE_TOO_LARGE` · `BAD_IMAGE`	Take or upload a photo first · under 8 MB · must be a JPEG or PNG.
`OCR_UNAVAILABLE`	Tesseract (or the handwriting model) is not installed – type or paste the address and use “Split into fields”.
`REASON_REQUIRED` · `LABEL_MISMATCH`	A reprint needs a reason · the scanned code is a different item – reprint.

B.4 Scans

Code	Meaning · what to do
`UNKNOWN_ITEM`	No letter with this ID (if the check character does not match, rescan).
`NOT_AUTHORISED`	Your role or office may not record this status for this letter.
`INVALID_TRANSITION`	Not an allowed next step from the current status.
`ACK_REQUIRED`	Delivered needs a signature, name or sheet number (or a written exception).
`EVENT_ID_CONFLICT` · `CONFLICT`	The event ID was used by someone else · the letter changed at the same moment – try again.
`LOCATION_MISMATCH` · `MOCK_LOCATION` · `FUTURE_TIME` · `ACK_EXCEPTION`	Reasons a scan was held – the postmaster reviews it.

APPENDIX C

Questions an examiner may ask

Short, honest answers you can build on.

Q What is new in your component compared with existing systems?

It joins four things that are usually separate: trilingual assisted address capture with a handwriting model and confidence flags, IoT weighing with a tariff engine that refuses to guess at band boundaries, location evidence judged with its uncertainty (policy L2), and a transactional outbox with offline-safe, idempotent scanning.

Q Why not let the OCR fill the form automatically?

Because even good readers make mistakes, especially on handwriting. The officer confirms every field; OCR output is stored separately as a suggestion and the corrections are measured.

Q How do you stop the previous letter’s weight being used again?

The firmware numbers every placement; the database allows each placement key (device + boot + placement) on only one draft or item. Test T03 proves it.

Q What happens at 29 g ± 2 g?

The quote is ”reweigh required”, because the true weight could be on either side of the 30 g step. The engine never silently rounds.

Q Why L2 instead of a simple radius?

A simple radius ignores GPS accuracy, so it wrongly rejects valid scans with a vague fix and accepts stale ones. L2 uses the probability of being inside; in simulation it gave 0 % false acceptance, 1.7 % mismatches on valid scans and 0 % stale acceptance.

Q Does a GPS point prove delivery?

No. It is supporting evidence. Delivery needs an acknowledgement, and messages never claim GPS proof.

Q What if the phone has no network?

The scan is saved on the phone first with a once-made event ID and uploaded later in order. The server is idempotent, so retries never create duplicates (1,000-replay test).

Q How can a message be sent for a scan that was rolled back?

It cannot. The message is written in the same single-document write as the scan and status (transactional outbox), so either all exist or none.

Q What does “provider accepted” mean?

Only that the e-mail service took the message – not that the sender read it. Timeouts become ”ambiguous” instead of being counted as sent.

Q How did you test the handwriting model fairly?

Model choices used validation lines only; the published test splits were read once after training and compared with Tesseract on the same lines: Sinhala CER 18.1 % vs 91 %, English 9.2 % vs 56 %. Tamil handwriting is untested and I say so.

Q How is personal data protected?

Office and role rules on every request, no personal data in codes, URLs, audit or exports, a secret token in the URL fragment, password hashes, a device key, and images excluded from git.

Q Is the scale accurate enough to charge customers?

It is a research demonstrator; accuracy is measured with reference-mass trials (target MAE ≤ 1 g). Official charging must use an approved scale, which the manual-weight path supports.

Q What would you do next?

Collect the consented envelope set and real Tamil handwriting, run the usability and location field trials, get the tariff approved, and add recipient OTP and push notifications.